



Universidad
Tecnológica
del Perú

FACULTAD DE INGENIERÍA DE SISTEMAS Y SOFTWARE

Integrador I: Sistemas-Software

**Sistema Web de control de préstamo de Libros de la
biblioteca “Biblioteca de Alejandría”**

Integrantes:

RIVAS GONZALES, GIANCARLO U18210506

AGUIRRE ALVARADO, RODOLFO JUNIOR U24201858

REATEGUI AGUILAR, JUNIOR ALEXIS U24245620

JARA OLIVA, ALONSO ANDRE U24246162

SOTO PAZ, GIANFRANCO JESUS ALEJANDRO U24205713

Lima - Perú

2026

INDICE

Contenido

INTRODUCCIÓN	6
CAPÍTULO 1 - ASPECTOS GENERALES	7
1.1. Descripción de la empresa	7
1.1.1. Misión.....	8
1.1.2. Visión	8
1.1.3. Rubro del negocio	8
1.2. Descripción del problema	8
1.2.1. Diagrama del árbol.....	9
1.2.2. Indicadores	10
1.3. Definición de objetivos.....	12
1.3.1. Objetivo general.....	12
1.3.2. Objetivos específicos	13
1.4. Alcances y limitaciones.....	13
1.4.1 Alcances.....	21
1.4.2 Limitaciones	22
1.5. Justificación	23
1.6. Estado del Arte	24
CAPÍTULO 2 – MARCO TEORICO.....	27
2.1 Marco teórico de programación orientada a objetos.....	27
2.1.1 Metodología	27
2.1.2 Modelo	32
2.1.2.1. Modelo Entidad-Relación (ER).....	32
2.1.2.2. Elementos de un diagrama ERD.....	32
2.1.2.3. Notación gráfica de relaciones.....	34
2.1.2.4. Cardinalidad.....	34
2.1.2.5. Diccionario de Datos	36
2.1.3. Patrones de la GUI	38
2.1.3.1. Patrón Formulario de Autenticación (Login Form Pattern)	39
2.1.3.2. Patrón de Validación de Entrada (Input Validation Pattern)	40
2.1.3.3. Patrón de Barra de Búsqueda (Search Pattern)	41
2.1.3.4. Patrón CRUD (Create, Read, Update and Delete).....	42

2.1.3.5. Patrón de Notificaciones (Notification Pattern).....	43
2.1.3.6. Patrón Dashboard.....	44
2.1.3.7. Patrón de Alertas (Alert Pattern)	45
2.1.4. Patrones de Desarrollo	46
2.1.4.1. Patrón Modelo–Vista–Controlador (MVC).....	46
2.1.4.2. Patrón Repository	47
2.1.4.3. Patrón DTO (Data Transfer Object).....	48
2.1.5 Lenguajes de programación.....	49
2.1.5.1. Lenguajes de programación.....	50
2.1.5.2. Frameworks e IDE	51
2.1.6 Gestor de base de datos.....	55
2.1.7 Herramientas adicionales.....	58
2.1.8 Diagrama de Clases	62
2.2 Marco teórico de las fórmulas y/o sistema	68
CAPITULO 3 - DESARROLLO DE LA SOLUCION.....	72
3.1. Modelo de proceso de negocios (diagrama As-Is).....	72
3.2. Prototipo interactivo.....	72
BIBLIOGRAFÍA	92

INDICE DE FIGURAS

Figura 1. Diagrama de árbol de la “Biblioteca de Alejandría”	9
Figura 2. Modelo de Gestión de Base de Datos.....	24
Figura 3. Interfaz del sistema web bibliotecario	26
Figura 4. Sistema de gestión de préstamos bibliotecarios	27
Figura 5. Metodología Scrum	31
Figura 6. Ejemplo Diagrama Entidad-Relación	35
Figura 7. Diccionario de datos.....	37
Figura 8. Formulario de Autenticación.....	39
Figura 9. Validación de Entrada	40

Figura 10. Barra de búsqueda.....	41
Figura 11. CRUD	42
Figura 12. Notificaciones.....	43
Figura 13. Dashboard.....	44
Figura 14. Alerta de comunicación	45
Figura 15. Diagrama del patrón Modelo Vista Controlador	47
Figura 16. Diagrama del Patrón Repository	48
Figura 17. Diagrama del Patrón DTO.....	49
Figura 18. Logo de JavaScript	50
Figura 19. Logo de SQL	51
Figura 20. Logo de Spring Boot	52
Figura 21. Logo de Thymeleaf.....	53
Figura 22. Logo de Bootstrap.....	54
Figura 23. Logo de Visual Studio Code.....	54
Figura 24. Logo de MySQL	56
Figura 25. Interfaz de Pencil Project	58
Figura 26. Interfaz de Bizagi Modeler.....	59
Figura 27. Interfaz de Lucidchart.....	59
Figura 28. Interfaz de Canva	60
Figura 29. Interfaz de Microsoft Word	61
Figura 30. Interfaz de Microsoft Excel	61
Figura 31. Diagrama de Clases del sistema (fragmento ilustrativo)	66

INDICE DE TABLAS

Tabla 1. Requerimientos Funcionales	13
---	----

Tabla 2. Requerimiento Funcional: Registro de préstamos	14
Tabla 3. Requerimiento Funcional: Registro de devoluciones	14
Tabla 4. Requerimiento Funcional: Consulta de disponibilidad de libros	15
Tabla 5. Requerimiento Funcional: Registro de reservas	15
Tabla 6. Requerimiento Funcional: Ampliación de tiempo de préstamo	15
Tabla 7. Requerimiento Funcional: Gestión de incidencias y sanciones	15
Tabla 8. Requerimiento Funcional: Consulta de historial de préstamos	16
Tabla 9. Requerimiento Funcional: Generación de reportes	16
Tabla 10. Requerimiento Funcional: Registro de usuarios	16
Tabla 11. Requerimiento Funcional: Inicio de sesión.....	17
Tabla 12. Requerimiento Funcional: Control de acceso por roles	17
Tabla 13. Requerimiento Funcional: Actualización de datos de usuarios	17
Tabla 14. Requerimiento Funcional: Eliminación de usuarios	17
Tabla 15. Requerimiento Funcional: Gestión de libros	18
Tabla 16. Requerimiento Funcional: Actualización de reservas	18
Tabla 17. Requerimiento Funcional: Cancelación de reservas.....	18
Tabla 18. Requerimiento Funcional: Eliminación de reservas	18
Tabla 19. Requerimientos No Funcional.....	19
Tabla 20. Requerimiento No Funcional: Disponibilidad del sistema	19
Tabla 21. Requerimiento No Funcional: Usabilidad y Diseño del Sistema	19
Tabla 22. Requerimiento No Funcional: Tiempo de respuesta	20
Tabla 23. Requerimiento No Funcional: Seguridad	20
Tabla 24. Requerimiento No Funcional: Integridad de datos	20
Tabla 25. Comparación de Metodologías	30

INTRODUCCIÓN

En la actualidad, la incorporación de herramientas tecnológicas en las organizaciones constituye un elemento importante para mejorar la gestión de la información, optimizar procesos y ofrecer servicios más eficientes. En el ámbito educativo y cultural, las bibliotecas cumplen un papel fundamental al facilitar el acceso al conocimiento y promover la lectura; sin embargo, cuando sus procesos administrativos continúan dependiendo de registros manuales, pueden presentarse dificultades para mantener un control preciso y oportuno de los recursos bibliográficos.

La gestión de préstamos y devoluciones requiere disponer de información organizada y actualizada que permita conocer la disponibilidad de los libros, identificar a los usuarios que utilizan los materiales y realizar un seguimiento adecuado de cada préstamo. Cuando estas actividades se realizan mediante cuadernos, anotaciones físicas o listas manuales, aumenta la posibilidad de registrar datos incompletos, duplicados o incorrectos, además de dificultar la consulta y actualización de la información.

En este contexto, surge la necesidad de mejorar la gestión de los préstamos de la Biblioteca de Alejandría mediante una solución tecnológica que permita organizar y controlar de manera más eficiente las actividades relacionadas con el uso de los materiales bibliográficos. La implementación de un sistema web permitirá centralizar la información y facilitar el registro, seguimiento y consulta de los préstamos y devoluciones, contribuyendo a una gestión más ordenada y confiable.

El proyecto contempla el desarrollo de un sistema web orientado al control de préstamos de libros, mediante el cual se podrá registrar la información correspondiente a los usuarios, materiales bibliográficos y operaciones realizadas. Asimismo, se busca facilitar la consulta de la disponibilidad de los libros y mantener un historial de las actividades efectuadas, proporcionando información que permita realizar un seguimiento adecuado de los recursos.

Para el desarrollo de la solución se plantea el uso de tecnologías web como HTML, CSS, Bootstrap y JavaScript para la construcción de la interfaz de usuario; Spring Boot con Java y Thymeleaf para la implementación de la aplicación en el lado del servidor; y MySQL para el almacenamiento y gestión de la información. La integración de estas tecnologías permitirá disponer de una solución orientada a mejorar la organización de los datos y la eficiencia de los procesos relacionados con los préstamos bibliotecarios.

De esta manera, el proyecto busca responder a las dificultades asociadas con la gestión manual de los préstamos, proporcionando una herramienta que permita reducir errores en los registros, facilitar el acceso a la información y mejorar el seguimiento de los materiales bibliográficos. La implementación de esta solución representa una oportunidad para modernizar la gestión de la Biblioteca de Alejandría y contribuir al ofrecimiento de un servicio más organizado, eficiente y confiable.

CAPÍTULO 1 - ASPECTOS GENERALES

1.1. Descripción de la empresa

La Biblioteca de Alejandría es una organización orientada al rubro educativo y cultural que se dedica a brindar servicios de acceso a la información, el conocimiento y la cultura mediante la gestión y administración de recursos bibliográficos. Su actividad principal se enfoca en la atención presencial para el préstamo y consulta de libros exclusivamente en sala, además de promover hábitos de lectura y apoyar activamente las tareas de formación e investigación de su comunidad. Para llevar a cabo sus operaciones, la institución dispone de un acervo ordenado por categorías, autores y áreas temáticas, y cuenta con un equipo de trabajo conformado por un Coordinador Bibliotecario y dos Encargados de Biblioteca, quienes reciben un flujo estimado de 150 a 200 solicitudes diarias de lectura.

Dentro de sus responsabilidades operativas, el personal debe registrar detalladamente la identificación de los usuarios, los datos de los textos solicitados y el tiempo de uso otorgado. Sin embargo, la gestión de la biblioteca ha mantenido un esquema de trabajo tradicional basado en procedimientos puramente manuales. Con la finalidad de resolver la lentitud en la atención y garantizar la disponibilidad real del

material, la Biblioteca de Alejandría busca fortalecer sus procesos a través de la implementación de una solución tecnológica centralizada que modernice el control del catálogo y facilite la atención de sus usuarios.

1.1.1. Misión

Nuestra misión es garantizar el acceso a la lectura y el conocimiento, ofreciendo a nuestros usuarios un acervo bibliográfico organizado, actualizado y de calidad. Nos comprometemos a brindar un entorno óptimo para el aprendizaje y la investigación, apoyados en servicios de préstamo en sala ágiles y eficientes que fomenten el desarrollo cultural e intelectual de la comunidad.

1.1.2. Visión

Ser una biblioteca reconocida por la eficiencia y confiabilidad de sus servicios, incorporando herramientas tecnológicas que permitan modernizar la gestión de los recursos bibliográficos y ofrecer a los usuarios un acceso organizado, accesible y de calidad a la información y al conocimiento.

1.1.3. Rubro del negocio

La Biblioteca de Alejandría pertenece al rubro de servicios educativos y culturales, específicamente al sector de gestión bibliotecaria y servicios de información. Sus actividades están orientadas a facilitar el acceso, consulta y préstamo de material bibliográfico, contribuyendo al aprendizaje, la lectura y el desarrollo cultural de sus usuarios.

1.2. Descripción del problema

En la Biblioteca de Alejandría, la administración del servicio de préstamos y devoluciones en sala depende en su totalidad del registro manual en cuadernos y fichas físicas. Esta modalidad genera serias dificultades operativas, ya que cuando un usuario solicita un ejemplar que no está en la estantería ni en los registros inmediatos, el personal debe revisar manualmente múltiples hojas de papel sin certeza de si el material está prestado, retenido fuera de plazo, mal ubicado o extraviado, lo cual ocasiona largas esperas y una atención ineficiente. Asimismo, durante el proceso de devolución, el cierre de la transacción suele limitarse a tachar o marcar a mano las anotaciones en el cuaderno, propiciando que libros devueltos

continúen figurando como pendientes o que no se registren formalmente los retrasos en los tiempos de entrega ni los daños físicos encontrados en el material. La ausencia de un registro digitalizado impide conocer el estado real del inventario en tiempo real, dificulta el seguimiento de las operaciones y limita la posibilidad de evaluar el comportamiento de uso de la comunidad.

El proceso operativo se desarrolla de manera secuencial iniciando cuando el usuario acude a la ventanilla para solicitar un libro en sala, presentando su documento de identidad y los datos del ejemplar requerido. En ese momento, el encargado de biblioteca realiza una búsqueda física en la estantería y revisa los cuadernos de registro; si el libro no se encuentra disponible, se le informa la situación al usuario y el flujo termina como un préstamo no realizado. Si el libro está disponible, el encargado procede a anotar manualmente en el cuaderno los datos del usuario, los datos del ejemplar y la hora límite de devolución (fijada en un máximo de tres horas), para luego entregar el texto al usuario, quien hace uso de este dentro de la sala de lectura.

Una vez concluida la lectura, el ciclo continúa cuando el usuario devuelve el libro en ventanilla y el encargado inspecciona su estado físico de conservación. Si el ejemplar presenta deterioros o si la devolución se realiza fuera del plazo establecido, el encargado anota la correspondiente observación o incidencia por mora en el registro manual. Finalmente, para concretar el cierre de la transacción, el encargado tacha la anotación previa en el cuaderno y reubica el texto en la estantería, concluyendo el proceso con el libro devuelto. Ante estas deficiencias, se hace evidente la necesidad de implementar un sistema web que centralice la información, optimice el seguimiento del material y garantice un control eficiente y confiable de los préstamos en la biblioteca.

1.2.1. Diagrama del árbol

Figura 1. Diagrama de árbol de la “Biblioteca de Alejandría”



Fuente: elaboración propia

1.2.2. Indicadores

Con el fin de mejorar el control de los préstamos de libros y garantizar una gestión eficiente dentro de la “Biblioteca de Alejandría”, el sistema propuesto incluirá indicadores clave que serán integrados dentro de la aplicación. Estos permitirán medir el desempeño de los procesos durante un periodo determinado (mensual), automatizar su cálculo y visualizar los resultados en tiempo real, facilitando la toma de decisiones.

A. Tiempo Promedio de Préstamo

Definición: Es el tiempo promedio que un libro permanece en uso por un usuario dentro de la sala, desde el momento del préstamo hasta su devolución, durante un periodo mensual.

Fórmula:

$$T_{promedio} = \frac{\sum(T_{devolucion} - T_{prestamo})}{N}$$

Donde:

- □□□□□□□□: Tiempo promedio de préstamo

- N : Número total de préstamos en el mes

Objetivo: Optimizar el tiempo de uso de los libros y mejorar la rotación del material bibliográfico.

B. Porcentaje de devoluciones fuera de plazo

Definición: Mide el porcentaje de préstamos que superan el tiempo máximo permitido (3 horas) durante un mes.

Fórmula:

$$\text{Retrasos Mensuales}(\%) = \frac{N_{\text{retrasos}}}{N_{\text{prestamos}}} \times 100$$

Donde:

- N_{retrasos} : Número de retrasos mensuales
- $N_{\text{prestamos}}$: Número de préstamos mensuales

Objetivo: Identificar el nivel de incumplimiento mensual de los usuarios y mejorar el control del tiempo de préstamo.

C. Índice de Precisión del Inventario

Definición: Mide el nivel de coincidencia exacta entre la cantidad de libros que el sistema informático marca como "Disponibles" y los libros físicos reales que se encuentran en los estantes.

Fórmula:

$$\text{Coincidencia}(\%) = \frac{N_{\text{libros-coincidentes}}}{N_{\text{total-auditado}}} \times 100$$

Donde:

- $\square_{\text{físicos}} - \square_{\text{registrados}}$: Cantidad de libros físicos que coinciden con el estado del sistema.
- $\square_{\text{auditado}}$: Total de libros revisados durante una auditoría mensual.

Objetivo: Eliminar los errores en los registros manuales y garantizar que el sistema refleje la realidad física de la biblioteca para saber siempre dónde está cada libro.

D. Tasa de Incidencias Mensual

Definición:

Mide la proporción de incidencias registradas (pérdidas) en relación con el total de préstamos durante un mes.

Fórmula:

$$IncidenciasMensuales(\%) = \frac{N_{\text{incidencias}}}{N_{\text{prestamos}}} \times 100$$

Donde:

- $N_{\text{incidencias}}$: Número de incidencias mensuales
- $\square_{\text{préstamos}}$: Número de préstamos mensuales

Objetivo: Cuantificar el volumen de material dañado o extraviado para implementar medidas correctivas, gestionar las sanciones correspondientes a los usuarios y garantizar la conservación del acervo bibliográfico.

1.3. Definición de objetivos

1.3.1. Objetivo general

Mejorar el control de los préstamos de libros en la "Biblioteca de Alejandría" mediante el desarrollo de un sistema web de gestión de préstamos, que permita optimizar el registro, seguimiento y control de los libros y usuarios, incrementando la eficiencia operativa, reduciendo errores y facilitando la toma de decisiones.

1.3.2. Objetivos específicos

- Implementar un sistema web que reemplace los registros manuales, permitiendo el almacenamiento digital de la información de préstamos en sala, devoluciones y usuarios de manera organizada y accesible.
- Desarrollar funcionalidades que permitan el seguimiento y control de los préstamos, incluyendo la verificación de disponibilidad, control estricto del tiempo de uso (máximo de 3 horas) y gestión de devoluciones.
- Reducir los errores en los registros mediante la automatización de los procesos utilizando interfaces, asegurando la integridad, precisión del inventario y actualización de la información registrada en el sistema.

1.4. Alcances y limitaciones

La definición de los alcances y limitaciones del sistema se realizará mediante la identificación de requerimientos funcionales y no funcionales, los cuales establecen las funcionalidades que el sistema debe cumplir y las condiciones bajo las cuales operará.

Para los requerimientos se han definido tres niveles de prioridad:

- Alta: Requerimientos indispensables para el funcionamiento del sistema.
- Media: Requerimientos importantes que mejoran el sistema.
- Baja: Requerimientos complementarios que no afectan el funcionamiento principal.

A. Requerimientos Funcionales

Tabla 1. Requerimientos Funcionales

Requerimientos Funcionales	
RF01	Registro de préstamos (exclusivo en sala)
RF02	Registro de devoluciones (con validación de estado físico)
RF03	Consulta de disponibilidad de libros

RF04	Registro de reservas
RF05	Ampliación de tiempo de préstamo (3 horas)
RF06	Gestión de incidencias y sanciones (por morosidad o daños)
RF07	Consulta de historial de préstamos
RF08	Generación de reportes
RF09	Registro de usuarios
RF10	Inicio de sesión
RF11	Control de acceso por roles
RF12	Actualización de datos de usuarios
RF13	Eliminación de usuarios
RF14	Gestión de libros (registro, actualización y eliminación)
RF15	Actualización de reservas
RF16	Cancelación de reservas (Lógico)
RF17	Eliminación de reservas (Físico)

Tabla 2. Requerimiento Funcional: Registro de préstamos

Número de requerimiento		RF01	
Nombre		Registro de préstamos	
Usuario	Encargado	Prioridad	Alta
El sistema permitirá registrar los préstamos de libros, incluyendo datos del usuario, libro, hora de préstamo y tiempo asignado (máximo 3 horas).			

Tabla 3. Requerimiento Funcional: Registro de devoluciones

Número de requerimiento		RF02	
Nombre		Registro de devoluciones	
Usuario	Encargado	Prioridad	Alta

El sistema permitirá registrar la recepción de libros, almacenando la hora exacta de retorno, exigiendo la validación obligatoria del estado físico del ejemplar y actualizando automáticamente su disponibilidad.

Tabla 4. Requerimiento Funcional: Consulta de disponibilidad de libros

Número de requerimiento		RF03	
Nombre		Consulta de disponibilidad de libros	
Usuario	Encargado / Usuario	Prioridad	Alta
El sistema permitirá verificar en tiempo real si un material bibliográfico se encuentra en estado "Disponible", "Prestado en sala" o "En mantenimiento".			

Tabla 5. Requerimiento Funcional: Registro de reservas

Número de requerimiento		RF04	
Nombre		Registro de reservas	
Usuario	Encargado	Prioridad	Alta
El sistema permitirá registrar reservas de libros que actualmente se encuentren en uso dentro de la sala, manteniendo un orden de atención secuencial para los usuarios que esperan.			

Tabla 6. Requerimiento Funcional: Ampliación de tiempo de préstamo

Número de requerimiento		RF05	
Nombre		Ampliación de tiempo de préstamo	
Usuario	Encargado	Prioridad	Media
El sistema controlará el tiempo de uso, emitiendo alertas visuales en el panel del encargado cuando un préstamo esté próximo a cumplir sus 3 horas límite o ya se encuentre en estado de morosidad.			

Tabla 7. Requerimiento Funcional: Gestión de incidencias y sanciones

Número de requerimiento		RF06	
Nombre		Gestión de incidencias y sanciones	
Usuario	Encargado	Prioridad	Alta
El sistema permitirá registrar incidencias (devoluciones con retraso, daños físicos o pérdida del libro) y aplicar bloqueos temporales o sanciones al perfil del usuario infractor.			

Tabla 8. Requerimiento Funcional: Consulta de historial de préstamos

Número de requerimiento		RF07	
Nombre		Consulta de historial de préstamos	
Usuario	Encargado	Prioridad	Media
El sistema permitirá visualizar el historial cronológico completo de los préstamos filtrado por usuario específico o por ejemplar bibliográfico.			

Tabla 9. Requerimiento Funcional: Generación de reportes

Número de requerimiento		RF08	
Nombre		Generación de reportes	
Usuario	Encargado	Prioridad	Media
El sistema generará reportes consolidados mensuales sobre la cantidad de préstamos realizados, incidencias registradas y el índice de rotación de los libros.			

Tabla 10. Requerimiento Funcional: Registro de usuarios

Número de requerimiento		RF09	
Nombre		Registro de usuarios	
Usuario	Administrador	Prioridad	Alta
El sistema permitirá registrar nuevos usuarios, almacenando sus datos personales y credenciales de acceso.			

Tabla 11. Requerimiento Funcional: Inicio de sesión

Número de requerimiento		RF10	
Nombre		Inicio de sesión	
Usuario	Administrador / Encargado / Usuario	Prioridad	Alta
El sistema permitirá a los usuarios registrados iniciar sesión mediante credenciales válidas para acceder al sistema.			

Tabla 12. Requerimiento Funcional: Control de acceso por roles

Número de requerimiento		RF11	
Nombre		Control de acceso por roles	
Usuario	Administrador	Prioridad	Alta
El sistema permitirá restringir el acceso a funcionalidades según el tipo de usuario.			

Tabla 13. Requerimiento Funcional: Actualización de datos de usuarios

Número de requerimiento		RF12	
Nombre		Actualización de datos de usuarios	
Usuario	Administrador	Prioridad	Alta
El sistema permitirá modificar y actualizar la información de los usuarios registrados, como nombre, identificación, contacto y estado.			

Tabla 14. Requerimiento Funcional: Eliminación de usuarios

Número de requerimiento		RF13	
Nombre		Eliminación de usuarios	
Usuario	Administrador	Prioridad	Alta

El sistema permitirá eliminar usuarios del sistema únicamente cuando se encuentren en estado inactivo. Un usuario será considerado inactivo cuando no haya registrado préstamos durante un periodo continuo de un mes.

Tabla 15. Requerimiento Funcional: Gestión de libros

Número de requerimiento		RF14	
Nombre		Gestión de libros	
Usuario	Administrador	Prioridad	Alta
El sistema permitirá registrar, actualizar y eliminar libros, incluyendo datos como título, autor, categoría y estado (disponible/no disponible).			

Tabla 16. Requerimiento Funcional: Actualización de reservas

Número de requerimiento		RF15	
Nombre		Actualización de reservas	
Usuario	Administrador	Prioridad	Alta
El sistema permitirá modificar los datos de una reserva, como la fecha y hora de atención o el usuario asociado, en caso de cambios necesarios.			

Tabla 17. Requerimiento Funcional: Cancelación de reservas

Número de requerimiento		RF16	
Nombre		Cancelación de reservas	
Usuario	Administrador / Encargado	Prioridad	Alta
El sistema permitirá cancelar (de manera lógica) reservas registradas cuando el usuario ya no requiera el libro o no se presente en el tiempo establecido.			

Tabla 18. Requerimiento Funcional: Eliminación de reservas

Número de requerimiento		RF17	
-------------------------	--	------	--

Nombre		Eliminación de reservas	
Usuario	Administrador	Prioridad	Alta
El sistema permitirá eliminar (de manera física) reservas registradas en la base de datos.			

B. Requerimientos No Funcionales

Tabla 19. Requerimientos No Funcional

Requerimientos No Funcionales	
RNF01	Disponibilidad del sistema
RNF02	Usabilidad y Diseño del Sistema
RNF03	Tiempo de respuesta
RNF04	Seguridad
RNF05	Integridad de datos

Tabla 20. Requerimiento No Funcional: Disponibilidad del sistema

Número de requerimiento	RNF01
Nombre	Disponibilidad del sistema
Prioridad	Alta
El sistema deberá estar disponible durante todo el horario de atención de la biblioteca, garantizando su estabilidad operativa para no interrumpir el servicio de préstamos en sala.	

Tabla 21. Requerimiento No Funcional: Usabilidad y Diseño del Sistema

Número de requerimiento	RNF02
Nombre	Usabilidad y Diseño del Sistema
Prioridad	Alta

El sistema contará con una interfaz intuitiva y fácil de usar para el personal. Estará construida empleando estándares web (HTML, CSS y Bootstrap) para garantizar una navegación limpia y eficiente.

Tabla 22. Requerimiento No Funcional: Tiempo de respuesta

Número de requerimiento	RNF03
Nombre	Tiempo de respuesta
Prioridad	Alta
Las operaciones y consultas en la plataforma web deberán ejecutarse y mostrar resultados en la pantalla en un tiempo máximo de 2 segundos, asegurando fluidez en la atención en ventanilla.	

Tabla 23. Requerimiento No Funcional: Seguridad

Número de requerimiento	RNF04
Nombre	Seguridad
Prioridad	Alta
El sistema protegerá la información mediante autenticación y control de accesos por roles a nivel de aplicación web. A nivel de base de datos, garantizará la seguridad mediante el almacenamiento de contraseñas cifradas y el uso de consultas parametrizadas para evitar accesos no autorizados e inyecciones de código	

Tabla 24. Requerimiento No Funcional: Integridad de datos

Número de requerimiento	RNF05
Nombre	Integridad de datos
Prioridad	Alta
El sistema garantizará que los datos registrados sean precisos y consistentes, utilizando MySQL como gestor de base de datos relacional para evitar pérdida de información, duplicidades o inconsistencias en el inventario.	

1.4.1 Alcances

- El sistema permitirá gestionar de manera centralizada la información de los usuarios y libros de la “Biblioteca de Alejandría”, facilitando el registro, actualización, consulta y administración de estos datos. Esto permitirá mantener la información organizada y disponible para los procesos relacionados con el préstamo de material bibliográfico.
- El sistema permitirá registrar y controlar los préstamos de libros exclusivamente en sala, almacenando información como el usuario, ejemplar solicitado, hora de inicio y tiempo asignado. El control del periodo máximo de 3 horas permitirá mejorar el cumplimiento de los tiempos establecidos por la biblioteca.
- El sistema permitirá registrar las devoluciones de los libros, incluyendo la validación de su estado físico y la actualización de su disponibilidad. Esto permitirá mantener un control más preciso sobre los ejemplares y reducir las inconsistencias entre la información registrada y el estado real de los libros.
- El sistema permitirá consultar en tiempo real la disponibilidad de los libros, identificando aquellos que se encuentren disponibles, prestados en sala o en mantenimiento. De esta manera, el personal podrá conocer rápidamente el estado de cada ejemplar y brindar una atención más eficiente a los usuarios.
- El sistema permitirá gestionar las reservas de libros que se encuentren actualmente en uso, manteniendo un orden de atención para los usuarios que esperan acceder al material. Asimismo, permitirá actualizar o cancelar las reservas cuando sea necesario.
- El sistema permitirá registrar incidencias relacionadas con retrasos, daños físicos o pérdida de libros, permitiendo asociarlas al usuario correspondiente y gestionar las sanciones o bloqueos temporales establecidos. Esto contribuirá a mejorar el control y conservación del material bibliográfico.
- El sistema permitirá generar reportes mensuales relacionados con los préstamos, incidencias y rotación de los libros. Estos reportes proporcionarán

información útil para evaluar el funcionamiento del servicio y apoyar decisiones relacionadas con la administración de los recursos bibliográficos.

- El sistema contará con mecanismos de autenticación y control de acceso según el rol del usuario, permitiendo proteger la información registrada y

restringir el acceso a las funcionalidades correspondientes. Además, se buscará mantener la integridad y consistencia de los datos almacenados mediante el uso de una base de datos relacional.

1.4.2 Limitaciones

- El sistema estará orientado específicamente a la gestión de préstamos de libros dentro de la “Biblioteca de Alejandría”, por lo que su funcionamiento se limitará a los procesos relacionados con el registro, control, devolución, reserva y seguimiento del material bibliográfico.
- Los préstamos gestionados por el sistema estarán restringidos al uso en sala, con un tiempo máximo establecido de 3 horas. No se contempla dentro del alcance actual la administración de préstamos externos o a domicilio.
- El correcto funcionamiento del sistema dependerá de que la información de los libros, usuarios, préstamos y devoluciones sea registrada y actualizada correctamente por el personal encargado de la biblioteca.
- El funcionamiento de la plataforma dependerá de la disponibilidad de los equipos informáticos, conexión de red y demás infraestructura necesaria para acceder al sistema web.
- El desarrollo se encontrará condicionado por el tiempo establecido para la ejecución del proyecto académico, por lo que no se contemplará la incorporación de funcionalidades adicionales que no formen parte de los requerimientos definidos.
- El sistema estará orientado al acceso mediante una plataforma web y no contemplará inicialmente el desarrollo de una aplicación móvil, debido al alcance establecido para esta versión del proyecto.

1.5. Justificación

El presente proyecto surge ante la necesidad de mejorar el control de los préstamos de libros en la "Biblioteca de Alejandría", debido a que actualmente estos procesos se realizan de manera manual mediante cuadernos y fichas físicas, lo que genera ineficiencias, errores en los registros y dificultades en el seguimiento de los libros y usuarios. Esta situación afecta directamente la organización interna y la calidad del servicio brindado, provocando demoras en la atención, libros extraviados y desconocimiento sobre la disponibilidad real de los materiales.

La implementación de un sistema web permitirá digitalizar y automatizar el proceso de gestión de préstamos en sala, facilitando el registro, control y seguimiento de la información en tiempo real. De esta manera, se reducirá la probabilidad de errores humanos, se optimizará el tiempo de atención y se mejorará la disponibilidad de la información, permitiendo conocer con precisión el estado de cada libro, quién lo tiene prestado y cuándo debe ser devuelto.

Asimismo, el sistema permitirá establecer un control adecuado del tiempo de préstamo (máximo de 3 horas en sala), así como gestionar reservas, ampliaciones e incidencias de manera organizada. Esto contribuirá a mejorar la rotación de los libros y a mantener un mejor control sobre el comportamiento de los usuarios, mediante el registro formal de sanciones en casos de retraso, daño o pérdida del material.

Por otro lado, la generación de reportes mensuales permitirá analizar el uso de los recursos bibliográficos, identificar los libros más solicitados y evaluar el desempeño del servicio, facilitando la toma de decisiones basada en datos, como la compra de nuevos ejemplares o la reubicación de materiales.

El proyecto se enfoca exclusivamente en la gestión de préstamos en sala, lo que permite desarrollar una solución específica y eficiente dentro del tiempo y recursos disponibles. De esta manera, se priorizan las funcionalidades esenciales que responden directamente a la problemática identificada, evitando complejidades innecesarias.

En consecuencia, la implementación de este sistema contribuirá a mejorar la eficiencia operativa, reducir errores, optimizar la gestión de la información y elevar la calidad del servicio, alineando a la Biblioteca de Alejandría con las necesidades

actuales de digitalización y ofreciendo a los usuarios un acceso más rápido y confiable a los materiales bibliográficos.

1.6. Estado del Arte

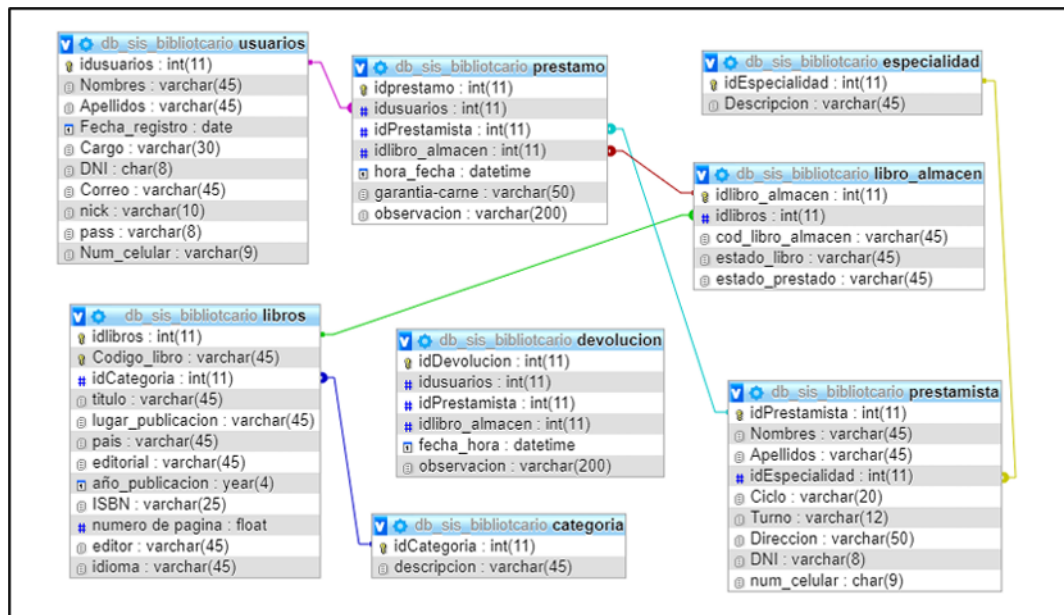
A. Sistema web de gestión bibliotecaria (Enfoque en Base de Datos)

En el trabajo desarrollado por Ruiz Loro (2021), titulado "Diseño de un sistema web para la gestión de biblioteca en la I.E. Federico Villarreal – Piura", se propone la implementación de un sistema web para mejorar la administración de los recursos bibliográficos. Este proyecto surge debido a la existencia de registros manuales, pérdida de información y dificultades en el control de préstamos.

Para el desarrollo del sistema, se utilizó una base de datos relacional en MySQL, la cual permitió organizar la información de usuarios, libros, préstamos y devoluciones de manera estructurada. Asimismo, el sistema facilita consultas rápidas y mantiene la integridad de los datos mediante relaciones entre tablas.

Este trabajo servirá como referencia para el diseño de la base de datos del presente proyecto, ya que proporciona un modelo estructurado para el almacenamiento de información, permitiendo definir tablas como usuarios, libros, préstamos, reservas e incidencias, asegurando la consistencia y confiabilidad de los datos. Esto es fundamental para el control de préstamos en sala que se requiere en la Biblioteca de Alejandría.

Figura 2. Modelo de Gestión de Base de Datos



Fuente: Ruiz Loro (2021)

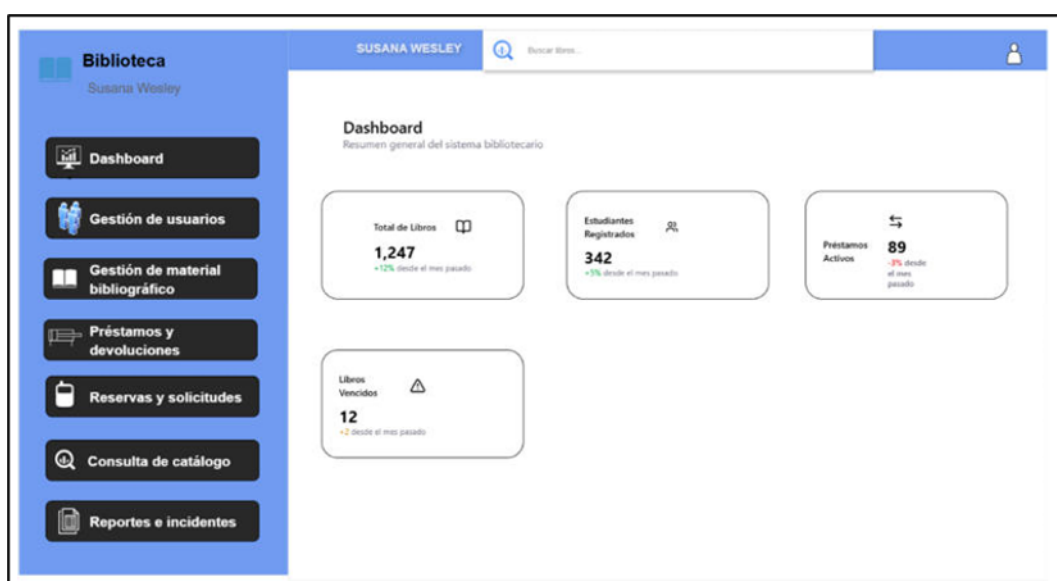
B. Diseño de un sistema web con software libre para la gestión de biblioteca (Enfoque en Interfaz de Usuario)

En el trabajo desarrollado por Gallardo Vásquez (2022), titulado "Diseño de un sistema web con software libre para la gestión de biblioteca", se propone una solución tecnológica enfocada en mejorar la interacción del usuario con el sistema mediante una interfaz clara y funcional.

El sistema fue desarrollado utilizando tecnologías como HTML, CSS y JavaScript, permitiendo diseñar interfaces intuitivas para el registro de préstamos, consultas de libros y gestión de usuarios. La organización visual de los módulos y la facilidad de navegación permiten que el sistema sea utilizado por personal sin conocimientos técnicos avanzados.

Este proyecto servirá como referencia para el diseño de la interfaz del sistema propuesto, ya que se tomará en cuenta la estructura de formularios, la distribución de elementos visuales y la experiencia de usuario, con el objetivo de desarrollar una interfaz amigable y eficiente para el encargado de la biblioteca, que facilite el rápido registro de préstamos en sala y la consulta de disponibilidad de libros.

Figura 3. Interfaz del sistema web bibliotecario



Fuente: Gallardo Vásquez (2022)

C. Desarrollo de un sistema de información para la gestión de préstamos en una biblioteca

El trabajo desarrollado por Chira Roque (2019), titulado "Desarrollo de un sistema informático para la gestión bibliotecaria", presenta una solución orientada específicamente al control de préstamos de libros dentro de una biblioteca, abordando problemas como el uso de registros manuales, la pérdida de información y la falta de seguimiento de los préstamos.

Este sistema fue desarrollado con el objetivo de automatizar los procesos de registro de usuarios, préstamos y devoluciones, permitiendo llevar un control más preciso sobre qué usuario tiene cada libro y en qué momento debe devolverlo. Asimismo, el sistema permite gestionar el historial de préstamos, facilitando el seguimiento del uso de los recursos bibliográficos.

El proyecto utiliza tecnologías web y bases de datos relacionales para almacenar la información de manera estructurada, garantizando la integridad de los datos y reduciendo errores en los registros. Además, incorpora funcionalidades como

consulta de disponibilidad de libros y generación de reportes, lo que mejora la toma de decisiones dentro de la biblioteca.

Este trabajo servirá como referencia principal para el desarrollo del presente proyecto, ya que está directamente relacionado con el control de préstamos de libros. A partir de este sistema se tomará como base la lógica del proceso de préstamo y devolución, el registro de usuarios y el seguimiento de los libros, adaptándolo a las necesidades específicas de la Biblioteca de Alejandría, incluyendo el control del tiempo de uso en sala (máximo 3 horas), la gestión de reservas, el registro de incidencias (daños, pérdidas) y la generación de reportes mensuales.

Figura 4. Sistema de gestión de préstamos bibliotecarios

REGISTRAR NUEVO PRESTAMO

Prestamo

Identificar Lector

Buscar:

CODIGO	NOMBRE	A. PATERNO	A. MATERNO	DNI
1	JESUS MARIA CHIRA	NAVARRO		78638628

Lector sel: JESUS MARIA CHIRA NAVARRO

Codigo:

Identificar Material Bibliográfico

Buscar:

CODIGO	TÍTULO	AUTOR	EDICIÓN	EJEMPLARES
1	EL CABALLE	ABRAHAN V...		5
2	FLACA	ANDRES CA...		3
3	LOS HERAL	CESAR VALL...		10

Material sel: LOS HERALDOS NEGROS - CESAR VALLEJO

Codigo:

Datos del préstamo

Cant. a Prestar: F. Prestamo: F. Devolución: Estado:

AGREGAR M. BIBLIOGRÁFICO

CODIGO	LECTOR	M. BIBLIOGRÁFICO	CANTIDAD
0	JESUS MARIA CHIRA NAVARRO	EL CABALLERO CARMELLO - ABRAH...	3
1	JESUS MARIA CHIRA NAVARRO	LOS HERALDOS NEGROS - CESAR VA...	5

REGISTRAR PRESTAMO

NUEVO PRESTAMO

CERRAR

Registros: 2

Fuente: Chira Roque (2019)

CAPÍTULO 2 – MARCO TEORICO

2.1 Marco teórico de programación orientada a objetos

2.1.1 Metodología

Para el desarrollo del presente proyecto se empleará la metodología ágil Scrum, debido a que permite gestionar el trabajo de manera iterativa e incremental, facilitando la adaptación a los cambios y la entrega continua de valor. Esta metodología resulta adecuada para el desarrollo del Sistema Web de control de préstamo de libros de la “Biblioteca de Alejandría”, ya que el proyecto requiere flexibilidad, validación

constante de las funcionalidades y una mejora progresiva del sistema conforme se avanza en su implementación.

Product Owner (PO)

El Product Owner es el representante de las necesidades del negocio, en este caso de la “Biblioteca de Alejandría”. Su función principal consiste en definir los requerimientos del sistema, priorizar las funcionalidades según su nivel de importancia (alta, media o baja) y asegurar que el producto final responda a las necesidades reales del encargado de la biblioteca y de los usuarios del servicio.

Scrum Master

El Scrum Master es responsable de velar porque el equipo de desarrollo aplique correctamente las prácticas de Scrum durante todo el proyecto. Asimismo, se encarga de identificar y eliminar los impedimentos que puedan afectar el avance del trabajo y de promover un ambiente de colaboración eficiente entre los integrantes del equipo.

Development Team

El equipo de desarrollo está conformado por los integrantes del proyecto: Rivas Gonzales, Giancarlo; Aguirre Alvarado, Rodolfo Junior; Reategui Aguilar, Junior Alexis; Jara Oliva, Alonso Andre; y Soto Paz, Gianfranco Jesus Alejandro, quienes se encargan del análisis, diseño, implementación y pruebas del sistema web. El trabajo se organiza en módulos definidos a partir de los requerimientos funcionales, tales como el registro de préstamos y devoluciones, la gestión de reservas, el control de usuarios y la generación de reportes.

Product Backlog

El Product Backlog es una lista priorizada de todas las funcionalidades que debe incluir el sistema, definidas y organizadas por el Product Owner según el valor que aportan a la gestión bibliotecaria. Para el Sistema Web de control de préstamo de libros de la Biblioteca de Alejandría, se consideran las siguientes funcionalidades principales:

- Registro de préstamos (exclusivo en sala)
- Registro de devoluciones con validación del estado físico del ejemplar
- Consulta de disponibilidad de libros

- Registro y gestión de reservas
- Ampliación de tiempo de préstamo
- Gestión de incidencias y sanciones por morosidad o daños
- Consulta de historial de préstamos
- Generación de reportes mensuales
- Registro, actualización y eliminación de usuarios
- Inicio de sesión y control de acceso por roles
- Gestión de libros (registro, actualización y eliminación)

Aplicación de Scrum en el proyecto

El enfoque de Scrum se basa en ciclos de trabajo iterativos denominados Sprints, así como en una serie de eventos que permiten organizar y mejorar continuamente el desarrollo del sistema.

a. Sprint

El Sprint es un ciclo de trabajo de corta duración (entre 2 y 3 semanas), en el cual se desarrolla un conjunto de funcionalidades del sistema. Cada Sprint tiene como objetivo entregar un incremento funcional del Sistema Web de control de préstamo de libros.

b. Sprint Planning (Planificación del Sprint)

Antes de iniciar cada Sprint, el equipo selecciona las funcionalidades del Product Backlog que serán desarrolladas durante el ciclo. Estas se dividen en tareas más pequeñas para facilitar su ejecución y asignación entre los integrantes del equipo.

c. Daily Scrum (Reunión diaria)

Durante el Sprint, el equipo realiza reuniones diarias de corta duración (máximo 15 minutos) para dar seguimiento al avance del trabajo. En estas reuniones se responde a las siguientes preguntas:

- ¿Qué se hizo el día anterior?
- ¿Qué se hará hoy?
- ¿Qué impedimentos existen?

d. Sprint Review (Revisión del Sprint)

Al finalizar cada Sprint, se presenta a los interesados el avance del sistema desarrollado. Esto permite validar las funcionalidades implementadas y recibir retroalimentación que contribuya a mejorar el producto.

e. Sprint Retrospective (Retrospectiva del Sprint)

Luego de la revisión, el equipo analiza el proceso de trabajo realizado durante el Sprint, identificando aspectos positivos y oportunidades de mejora que permitan optimizar el rendimiento en los siguientes ciclos.

f. Sprint Backlog

El Sprint Backlog es el conjunto de tareas seleccionadas del Product Backlog para ser desarrolladas durante un Sprint específico. Este documento sirve como guía de trabajo diario para el equipo de desarrollo.

Tabla 25. Comparación de Metodologías

Concepto	Waterfall	Agile	Scrum	Kanban
Estructura	Muy rígida	Flexible	Semiflexible	Muy flexible
Ciclos de trabajo	Fases largas	Iteraciones	Sprints fijos	Flujo continuo
Ideal para	Proyectos predecibles	Entornos cambiantes	Desarrollo de producto	Flujo continuo
Tamaño de equipo	Cualquiera	3-15 personas	3-9 personas	1-10 personas
Curva de aprendizaje	Baja	Media	Media-alta	Muy baja
Reuniones	Pocas	Frecuentes	Muy estructuradas	Ninguna obligatoria
Visibilidad del avance	Por fases	Por iteraciones	Por sprints	En tiempo real

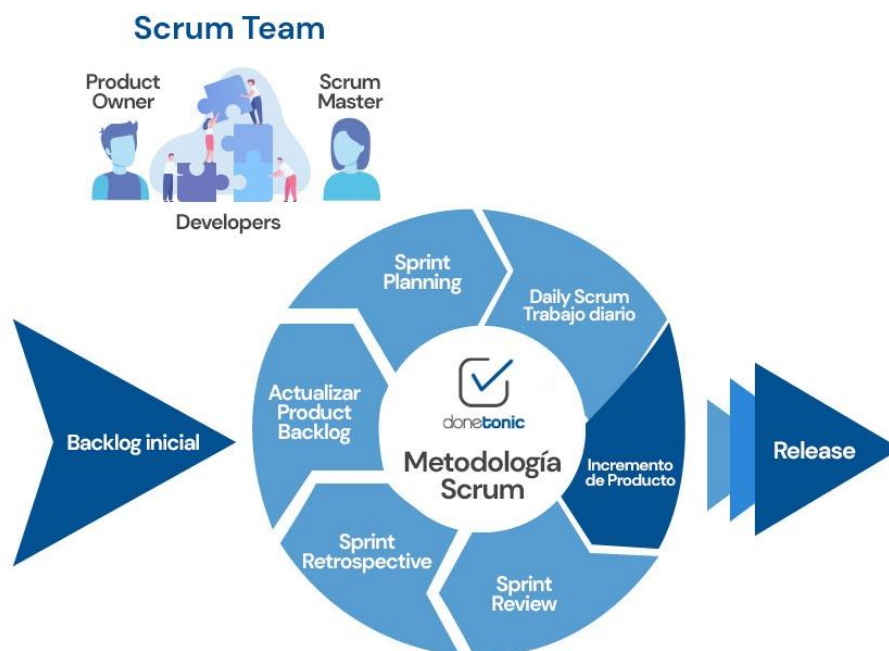
Recomendado para	Proyectos puntuales	Proyectos iterativos	Con adaptación	Gestión del día a día
-------------------------	---------------------	----------------------	----------------	-----------------------

Fuente: Red Ninja (2024). Metodologías de gestión de proyectos.

Justificación

Se seleccionó la metodología Scrum para el desarrollo del Sistema Web de control de préstamo de libros de la Biblioteca de Alejandría debido a que permite gestionar el proyecto de manera iterativa e incremental mediante Sprints, facilitando la planificación, el control del avance y la entrega progresiva de funcionalidades. A diferencia de metodologías con un flujo continuo de trabajo, como Kanban, Scrum ofrece una estructura más organizada, con roles y eventos claramente definidos, lo cual resulta más adecuado para un proyecto académico con objetivos específicos y un tiempo de desarrollo limitado. De esta manera, se logra una mejor coordinación entre los integrantes del equipo, una distribución eficiente de las tareas y una mayor probabilidad de cumplir con los objetivos planteados dentro del plazo establecido para el proyecto.

Figura 5. Metodología Scrum



Donetonic (s.f)

La figura presenta un esquema visual en forma de rueda que ilustra el funcionamiento de la metodología Scrum.

2.1.2 Modelo

2.1.2.1. Modelo Entidad-Relación (ER)

El Modelo Entidad-Relación es una herramienta conceptual utilizada en la ingeniería de software para representar la estructura lógica de una base de datos. Se basa en la percepción de un mundo real que consiste en una colección de objetos básicos, denominados entidades, y de las relaciones entre estos objetos. Su funcionamiento principal es servir como un plano arquitectónico que permite visualizar cómo se almacenan, vinculan y organizan los datos antes de su implementación física en un sistema gestor de bases de datos (SGBD).

2.1.2.2. Elementos de un diagrama ERD

Para la construcción de un modelo relacional, se utilizan los siguientes componentes fundamentales:

Entidad:

Las entidades representan objetos o conceptos del mundo real sobre los cuales se desea almacenar información.

Tipos de entidades:

- **Entidad fuerte:** Existe de manera independiente.
Ejemplo: Usuario, Libro
- **Entidad débil:** Depende de otra entidad para existir.
Ejemplo: Detalle de préstamo

Además de las entidades fuertes y débiles, el modelo Entidad-Relación cuenta con una extensión conocida como Modelo Entidad-Relación Extendido (EER), que incorpora el concepto de generalización/especialización. Esta técnica permite definir una entidad general (superclase) de la cual se derivan entidades más específicas (subclases), donde cada subclase hereda los atributos y relaciones de la superclase y añade sus propios atributos particulares. Bagui (2009) señala que el modelo ER básico se ha extendido para incluir generalizaciones y especializaciones, lo cual permite representar con mayor

precisión entidades del mundo real que comparten características comunes pero también poseen particularidades propias. Una especialización puede clasificarse como total (todo miembro de la superclase pertenece obligatoriamente a alguna subclase) o parcial (puede haber miembros que no pertenezcan a ninguna subclase), y como disjunta (un miembro pertenece a una sola subclase a la vez) o solapada (puede pertenecer a varias subclases simultáneamente).

Relación:

Es una asociación o vínculo entre dos o más entidades. Describe cómo las entidades interactúan entre sí dentro del sistema.

Tipos de relación:

- **Binarias:** Entre dos entidades
- **Ternarias:** Entre tres entidades
- **N-arias:** Más de tres entidades

Atributo:

Son las propiedades o características que describen a una entidad.

Tipos de atributos:

- **Simples:** No se pueden dividir (ej. Edad).
- **Compuestos:** Se pueden dividir en subpartes (ej. Nombre completo en nombre y apellido).
- **Monovalentes:** Tienen un solo valor para una entidad.
- **Multivalentes:** Pueden tener varios valores (ej. Teléfonos).
- **Derivados:** Su valor se calcula a partir de otros (ej. Edad a partir de la fecha de nacimiento).

Tipos de claves:

- **Clave Primaria (Primary Key):** Atributo único que identifica unívocamente a cada instancia de una entidad.

- **Clave Foránea (Foreign Key):** Atributo que establece un vínculo entre dos tablas al hacer referencia a la clave primaria de otra.

2.1.2.3. Notación gráfica de relaciones

En esta metodología, los elementos del diagrama se representan mediante una simbología técnica que facilita la transición al diseño físico de las tablas:

- **Entidades (Rectángulos):** Se representan como cajas que contienen el nombre de la entidad en la parte superior.
- **Atributos:** Se listan dentro del cuerpo del rectángulo de la entidad. Los atributos clave (Primary Keys) se ubican generalmente en la parte superior o se marcan con un símbolo distintivo (como una llave o las siglas PK).
- **Relaciones (Líneas):** Se representan mediante líneas que conectan las entidades. El tipo de relación se identifica por los símbolos en los extremos de la línea, cerca de la unión con la entidad.

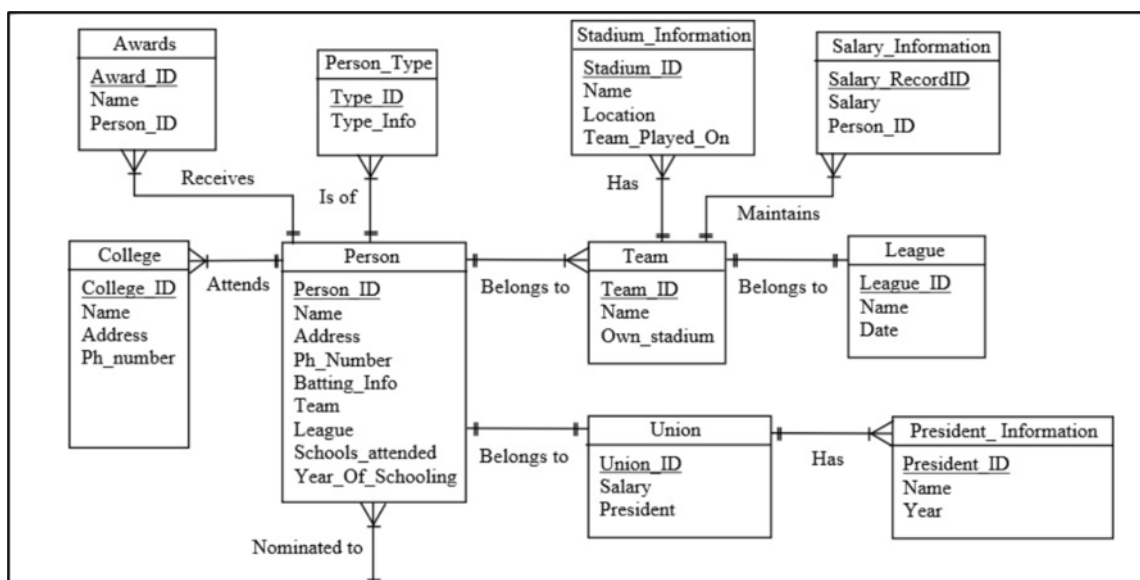
2.1.2.4. Cardinalidad

La cardinalidad define los límites de participación (mínimos y máximos) entre las entidades. Según la notación de Pata de Gallo, se utilizan los siguientes símbolos:

- **Uno a uno:** Se representa gráficamente con dos líneas perpendiculares intersecando la línea de relación en ambos extremos. Indica que un registro de la entidad de origen debe estar asociado de forma obligatoria y exacta a un único registro de la entidad de destino, y viceversa.
 - **Ejemplo:** La relación entre las entidades Préstamo y Comprobante de Emisión. Cada préstamo registrado genera obligatoriamente un único comprobante, y ese comprobante pertenece exclusivamente a ese préstamo específico.
- **Uno a muchos:** Se representa gráficamente colocando dos líneas perpendiculares en el extremo de la entidad "uno" y una bifurcación en forma de "pata de gallo" (que puede incluir un círculo o línea interna según la opcionalidad) en el extremo de la entidad "muchos". Establece que un registro de la primera entidad se conecta con múltiples registros de la segunda.

- **Ejemplo:** La relación entre Usuario (Lector) y Préstamo. En el extremo del Usuario se visualizan las líneas perpendiculares y en el extremo de Préstamo la pata de gallo; esto indica que un alumno puede realizar muchos préstamos a lo largo del ciclo, pero cada préstamo pertenece a ese único usuario.
- **Muchos a Uno:** Es la inversión de la relación anterior analizada desde la perspectiva opuesta. Se representa gráficamente colocando la bifurcación en forma de "pata de gallo" en el extremo de la entidad origen ("muchos") y dos líneas perpendiculares intersecando la línea en el extremo de la entidad destino ("uno").
 - **Ejemplo:** La relación entre Ejemplar de Libro y Categoría. Múltiples registros de libros físicos diferentes (muchos) están asignados en su extremo a una única categoría temática o sección de la biblioteca (uno).
- **Muchos a Muchos:** Se representa gráficamente colocando una bifurcación en forma de "pata de gallo" en ambos extremos de la línea de relación. Indica que múltiples registros de la entidad de origen pueden asociarse con múltiples registros de la entidad de destino.
 - **Ejemplo:** La relación entre Préstamo y Libro (cuando un alumno se lleva varios textos a la vez). Un solo préstamo puede contener muchos libros concurrentes, y a su vez, un título de libro específico puede haber sido parte de muchos préstamos diferentes a lo largo del tiempo.

Figura 6. Ejemplo Diagrama Entidad-Relación



Fuente: Chegg (s.f.)

2.1.2.5. Diccionario de Datos

El diccionario de datos es un inventario centralizado y detallado de todos los elementos que componen la base de datos del sistema. Su propósito es proporcionar una definición lógica y física de los datos, eliminando la ambigüedad entre los desarrolladores y asegurando que todos los componentes sigan las mismas reglas de negocio y restricciones técnicas.

Para que el diccionario sea efectivo en un entorno de desarrollo profesional, cada entrada debe contener los siguientes metadatos detallados:

a. Nombre del Campo (Atributo):

Es el identificador técnico o nombre de la columna en la tabla. Debe seguir una nomenclatura estándar (ej. id_usuario, fec_prestamo).

b. Descripción Funcional:

Explica el propósito del dato y qué representa en el contexto del negocio (ej. "Almacena la fecha única en que se realiza la solicitud del libro").

c. Tipo de Dato y Longitud:

Define la naturaleza técnica del valor y su capacidad de almacenamiento.

- Numéricos: INT, BIGINT, DECIMAL (para precisión).
- Cadenas: VARCHAR(n) para longitud variable o CHAR(n) para longitud fija.
- Temporales: DATE, DATETIME o TIMESTAMP.

d. Restricciones de Integridad (Constraints):

Define las reglas que el dato debe cumplir obligatoriamente:

- Nullability: Determina si el campo permite valores nulos (NULL) o si es obligatorio (NOT NULL).
- Claves (Key): Identifica si el campo es una Clave Primaria (PK), Clave Foránea (FK) o si posee un índice único (UNIQUE).
- Valores por defecto (Default): Define un valor automático si no se ingresa uno (ej. estado = 'activo').

e. Origen o Procedencia:

Indica si el dato es ingresado directamente por el usuario, generado automáticamente por el sistema (como un ID autoincremental) o calculado a partir de otros campos (dato derivado).

Figura 7. Diccionario de datos

Tabla Clientes

	Columna	Tipo de Dato	Descripción	Null
PK	ID_cliente	int	Numero de identificación para clientes	
	nombre_completo	varchar(500)	Nombre complete del cliente	
	direccion	text	Dirección de cliente	
	telefono	varchar(50)	Telefono de contacto	
	descuento_especial	decimal(18,2)	Descuento applicable al cliente	
	descuento	bit	Indicador si aplica descuento	

Tabla Compras

	Columna	Tipo de Dato	Descripción	Null
PK	ID_compra	int	Codigo de la compra	
	codigo_interno	varchar(50)	Codigo interno de identificacion	
	Fecha	datetime	Fecha de compra	
FK	ID_tipo	int	Numero de identificación de tipo	
FK	ID_proveedor	int	Numero de identificación para el proveedor	
FK	ID_usuario	int	Numero de identificación del usuario	

Tabla Compra_detalle

	Columna	Tipo de Dato	Descripción	Null
PK	ID_detalle	int	Numero de identificación al detalle	
FK	ID_compra	int	Numero de identificación para la compra	
FK	ID_producto	int	Numero de identificación del producto	
	cantidad	int	Cantidad de producto	
	cantidad_dec	decimal(18,2)	Cantidad en decimal	
FK	ID_unidad	int	Numero de identificación	
	costo	decimal(18,2)	Costo del producto	

Tabla Facturas

	Columna	Tipo de Dato	Descripción	Null
PK	ID_factura	int	Codigo de factura	
	codigo_interno	varchar(50)	Codigo interno de identificacion	
	fecha	datetime	Fecha de facturacion	
FK	ID_tipo	int	Numero de identificación de tipo	
FK	ID_cliente	int	Numero de identificación para clientes	
FK	ID_usuario	int	Numero de identificación del usuario	
	credito	bit	Indicador de disponibilidad de credito	
	fecha_vencimiento_credito	datetime	Fecha limite para pago de credito	
	total	decimal(18,2)	Total a pagar	
	total_pagado	decimal(18,2)	Total pagado	

Fuente: MySQLYa (s.f.)

2.1.3. Patrones de la GUI

Los patrones de GUI son soluciones reutilizables utilizadas para organizar y mejorar la interacción entre los usuarios y una interfaz gráfica. Estos patrones permiten estructurar elementos como formularios, botones, menús, búsquedas y mensajes de manera que sean fáciles de comprender y utilizar, contribuyendo a que la navegación dentro del sistema sea más clara y ordenada.

Para el desarrollo del Sistema Web de Control de Préstamos de la Biblioteca de Alejandría se utilizarán diferentes patrones de interfaz gráfica, debido a que el sistema

requiere gestionar una cantidad importante de información relacionada con usuarios, libros, préstamos, devoluciones e incidencias. La aplicación de estos patrones permitirá mejorar la experiencia del usuario y facilitar la realización de las diferentes operaciones dentro del sistema.

2.1.3.1. Patrón Formulario de Autenticación (Login Form Pattern)

Este patrón es utilizado principalmente en sistemas que requieren controlar el acceso de los usuarios y proteger la información almacenada. Consiste en una interfaz donde el usuario debe ingresar sus credenciales, generalmente un nombre de usuario o correo electrónico junto con una contraseña, para poder acceder al sistema. También puede incluir opciones adicionales como recuperar la contraseña o recordar la sesión. Su diseño suele ser sencillo y claro, permitiendo que el proceso de inicio de sesión sea rápido y fácil de comprender para los usuarios.

El patrón de formulario de autenticación se utiliza en el Sistema Web de Control de Préstamos de la Biblioteca de Alejandría porque permite verificar la identidad de las personas que ingresan al sistema y controlar el acceso a sus diferentes funcionalidades. De esta manera, solo los usuarios autorizados podrán realizar operaciones como gestionar libros, registrar préstamos, devoluciones o consultar información del sistema. Además, contribuye a mejorar la seguridad y evitar el acceso no autorizado a la información registrada en la biblioteca.

Figura 8. Formulario de Autenticación

LOGIN TO YOUR ACCOUNT

Email

Password

☐ Remember me [Forgot Password?](#)

SIGN IN

Don't Have an Account? [REGISTER NOW](#)

Fuente: CodexWorld, 2022

2.1.3.2. Patrón de Validación de Entrada (Input Validation Pattern)

El patrón de validación de entrada permite verificar que la información ingresada por los usuarios cumpla con las condiciones necesarias antes de ser procesada o almacenada en el sistema. Este patrón puede comprobar, por ejemplo, que los campos obligatorios hayan sido completados, que los datos tengan el formato correcto o que no se ingresen valores inválidos. Cuando existe algún error, el sistema muestra un mensaje para que el usuario pueda corregir la información antes de continuar con la operación.

Este patrón será utilizado en los diferentes formularios del Sistema Web de Control de Préstamos de la Biblioteca de Alejandría, especialmente en el registro y actualización de usuarios, libros, préstamos y reservas. Su implementación permitirá reducir errores en el ingreso de datos y evitar que se almacene información incompleta o incorrecta en la base de datos, contribuyendo también a mantener una información más organizada y confiable dentro del sistema.

Figura 9. Validación de Entrada

VALIDACION FORMULARIOS

Usuario	Nombre
john123 ✖	Carlos Arturo ✔
El usuario tiene que ser de 4 a 16 dígitos y solo puede contener numeros, letras y guion bajo.	
Contraseña	Repetir Contraseña
•••••••• ✔	•••••••• ✖
Ambas contraseñas deben ser iguales.	

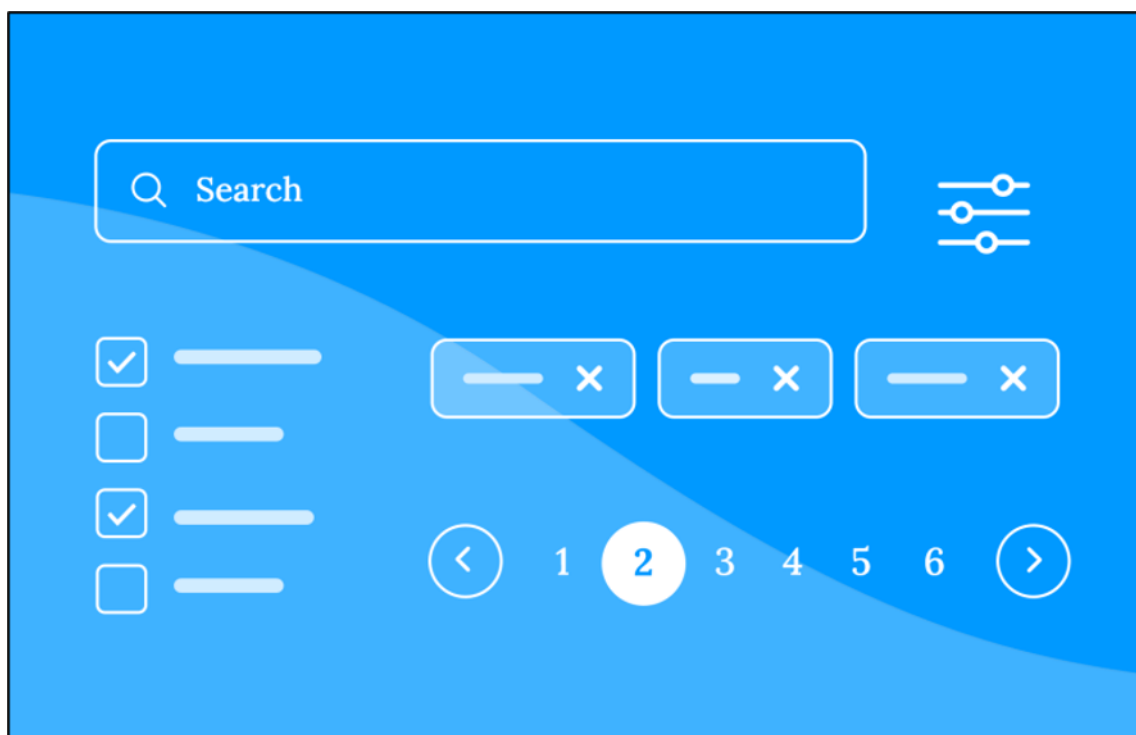
Fuente: P. Puig (2026)

2.1.3.3. Patrón de Barra de Búsqueda (Search Pattern)

El patrón de barra de búsqueda permite a los usuarios encontrar información específica dentro de un sistema mediante el ingreso de palabras, nombres o algún otro criterio relacionado con los datos almacenados. Generalmente consiste en un campo de texto acompañado de un botón o ícono de búsqueda, mostrando posteriormente los resultados que coinciden con la información ingresada. Este patrón resulta especialmente útil en sistemas que manejan una gran cantidad de registros.

En el Sistema Web de Control de Préstamos de la Biblioteca de Alejandría, este patrón permitirá facilitar la búsqueda de libros, usuarios y registros relacionados con los préstamos. Por ejemplo, el personal podrá localizar rápidamente un libro mediante su título, autor o código, evitando tener que revisar manualmente todos los registros disponibles. De esta manera, se podrá acceder a la información de una forma más rápida y mejorar la eficiencia durante las operaciones realizadas en la biblioteca.

Figura 10. Barra de búsqueda



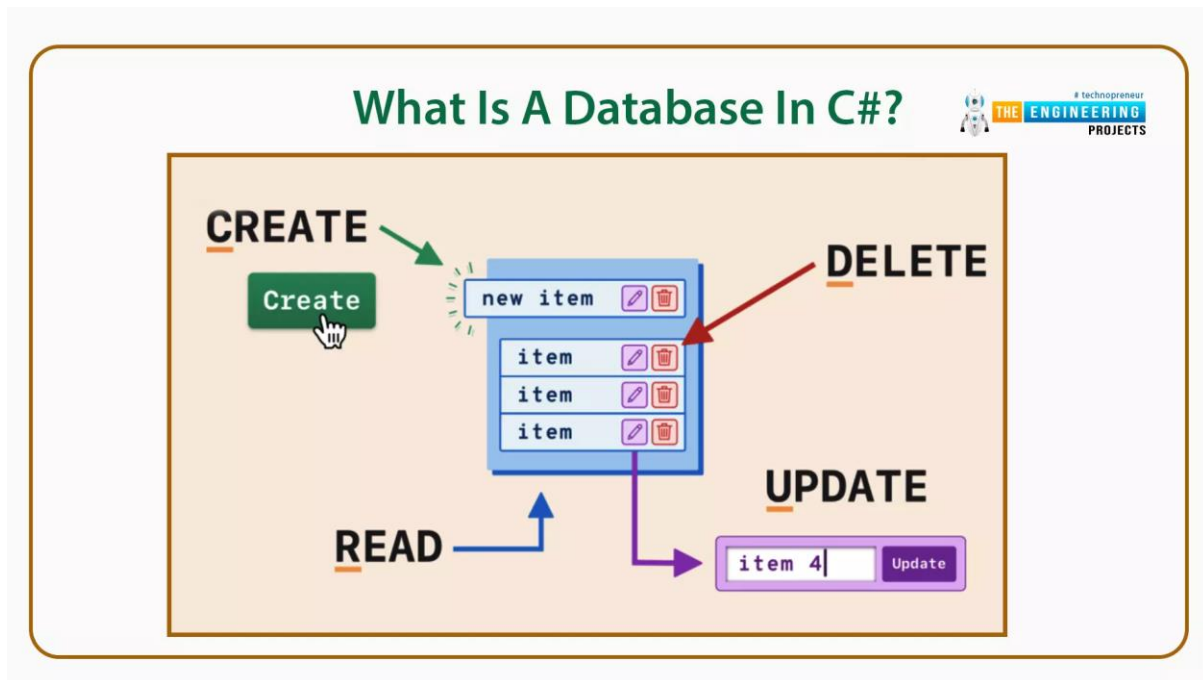
Fuente: Justinmind (2025)

2.1.3.4. Patrón CRUD (Create, Read, Update and Delete)

El patrón CRUD representa las cuatro operaciones básicas que normalmente se realizan sobre la información almacenada dentro de un sistema: crear, consultar, actualizar y eliminar registros. Estas operaciones permiten gestionar los datos de manera organizada y son utilizadas principalmente en sistemas que requieren administrar diferentes tipos de información mediante formularios, tablas y opciones de acción.

Este patrón será aplicado en el Sistema Web de Control de Préstamos de la Biblioteca de Alejandría para gestionar la información relacionada con los usuarios, libros y otros registros necesarios para el funcionamiento del sistema. Por ejemplo, será posible registrar nuevos libros, consultar la información existente, actualizar los datos cuando sea necesario y eliminar registros según las funciones permitidas. La utilización del patrón CRUD permitirá mantener una gestión más ordenada y facilitar las operaciones administrativas dentro de la biblioteca.

Figura 11. CRUD



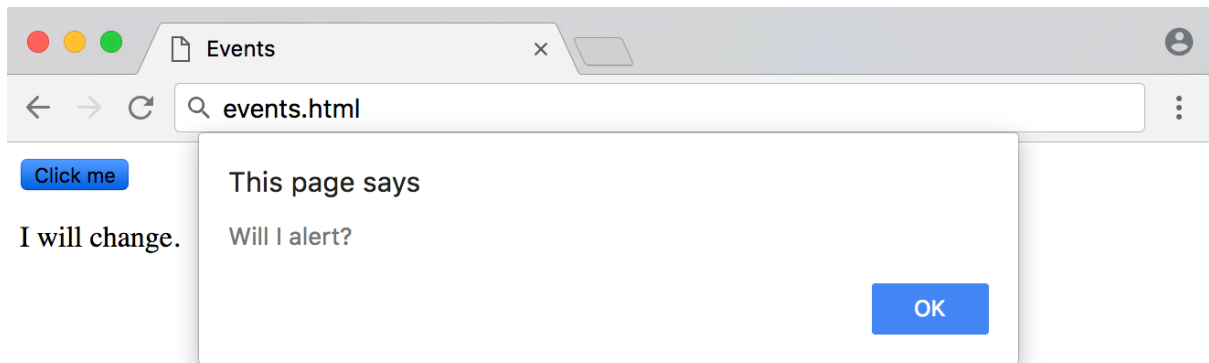
Fuente: Nasir, S. Z. (2021).

2.1.3.5. Patrón de Notificaciones (Notification Pattern)

El patrón de notificaciones consiste en mostrar mensajes al usuario después de que se produce una acción dentro del sistema. Estas notificaciones pueden informar que una operación se realizó correctamente, advertir sobre algún problema o comunicar información importante relacionada con una determinada actividad. Generalmente se presentan mediante mensajes temporales, ventanas emergentes o avisos visibles dentro de la interfaz.

En el Sistema Web de Control de Préstamos de la Biblioteca de Alejandría, este patrón permitirá mantener informado al usuario sobre el resultado de las operaciones realizadas. Por ejemplo, el sistema podrá mostrar mensajes cuando se registre correctamente un préstamo, se devuelva un libro, se actualice la información de un usuario o exista algún error durante una operación. Esto permitirá que los usuarios comprendan rápidamente lo que está ocurriendo dentro del sistema y tengan una mejor interacción con la aplicación.

Figura 12. Notificaciones



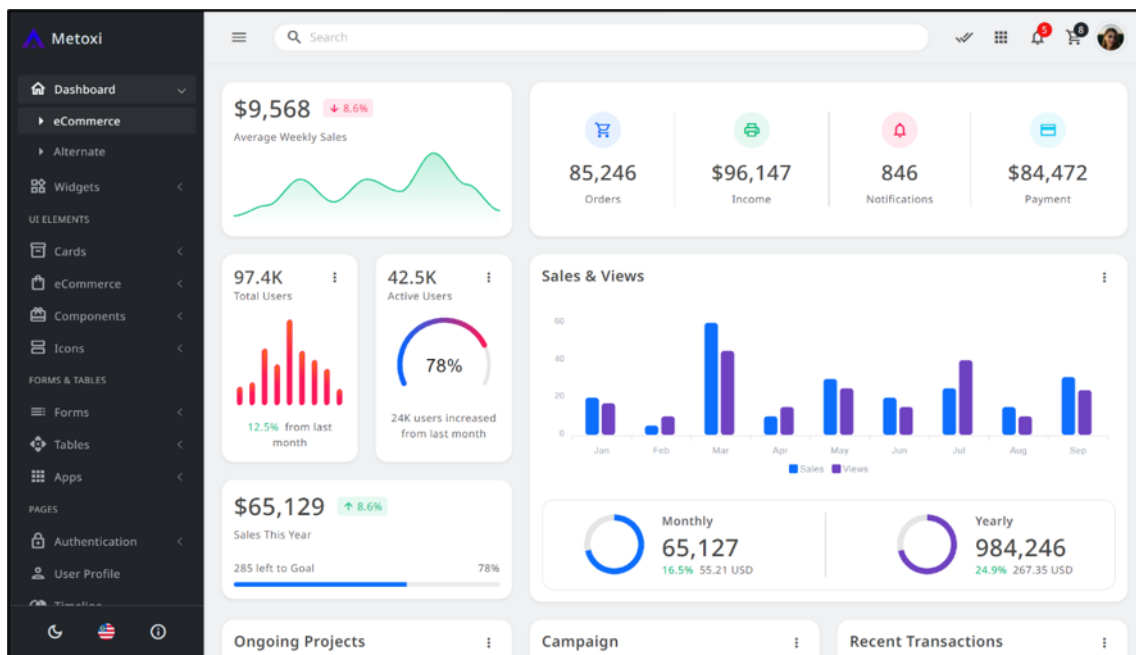
Fuente: Rascia, T., & Tagliaferri, L. (2021).

2.1.3.6. Patrón Dashboard

El patrón Dashboard consiste en una interfaz que permite mostrar de manera resumida la información más importante de un sistema mediante indicadores, tarjetas, gráficos y otros elementos visuales. Su principal función es facilitar la visualización rápida de los datos relevantes, evitando que el usuario tenga que revisar grandes cantidades de información para conocer la situación general de los procesos.

Este patrón será utilizado en el Sistema Web de Control de Préstamos de la Biblioteca de Alejandría para mostrar información relacionada con la gestión de los libros y préstamos. Por medio del Dashboard se podrán visualizar datos como la cantidad de libros disponibles, préstamos activos, devoluciones realizadas, incidencias registradas u otros indicadores importantes para la biblioteca. Esto facilitará el seguimiento de las actividades y permitirá que el personal pueda consultar rápidamente la información principal del sistema.

Figura 13. Dashboard



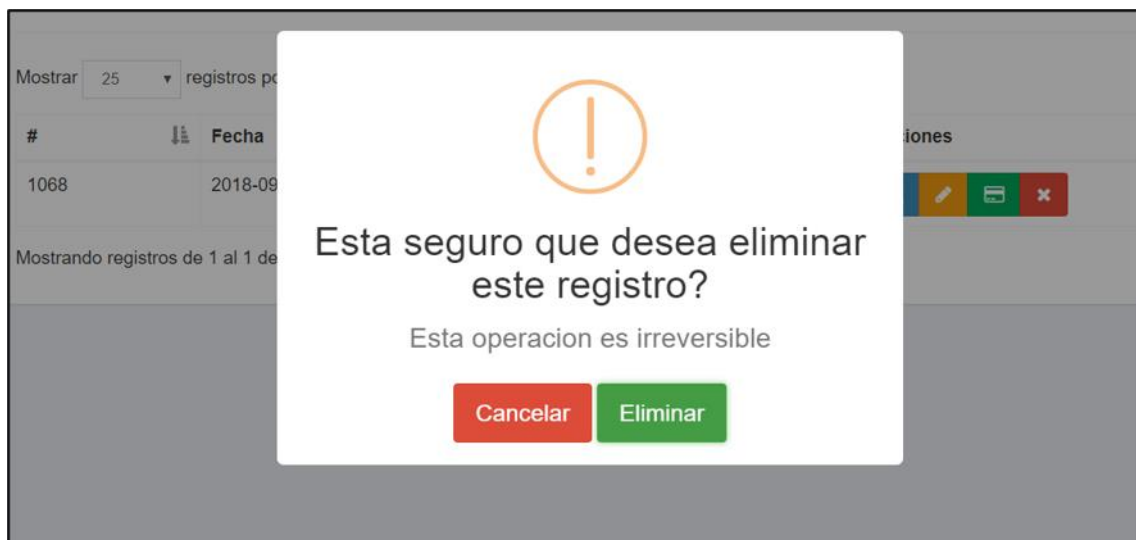
Fuente: Codervent. (s. f.).

2.1.3.7. Patrón de Alertas (Alert Pattern)

El patrón de alertas permite comunicar al usuario situaciones que requieren atención dentro del sistema. A diferencia de una notificación común, una alerta generalmente se utiliza cuando existe alguna condición importante, un posible error o una situación que debe ser revisada. Estas pueden mostrarse mediante mensajes visuales, ventanas emergentes o indicadores dentro de la interfaz, dependiendo de la importancia de la información.

En el Sistema Web de Control de Préstamos de la Biblioteca de Alejandría, este patrón permitirá advertir sobre situaciones relacionadas con los préstamos y el estado de los libros. Por ejemplo, se podrán mostrar alertas cuando un préstamo esté próximo a vencer, cuando se haya superado el tiempo establecido para la devolución o cuando se registre alguna incidencia relacionada con un libro. La utilización de este patrón permitirá que el personal pueda identificar estas situaciones con mayor rapidez y tomar las acciones necesarias.

Figura 14. Alerta de comunicación



Fuente: Pontificia Universidad Católica del Perú. (2022)

2.1.4. Patrones de Desarrollo

Los patrones de desarrollo son soluciones utilizadas durante la construcción de software para resolver problemas comunes relacionados con la organización y estructura de una aplicación. Su aplicación permite separar las diferentes responsabilidades del sistema, mejorar la reutilización del código y facilitar su mantenimiento a medida que el proyecto crece o requiere modificaciones.

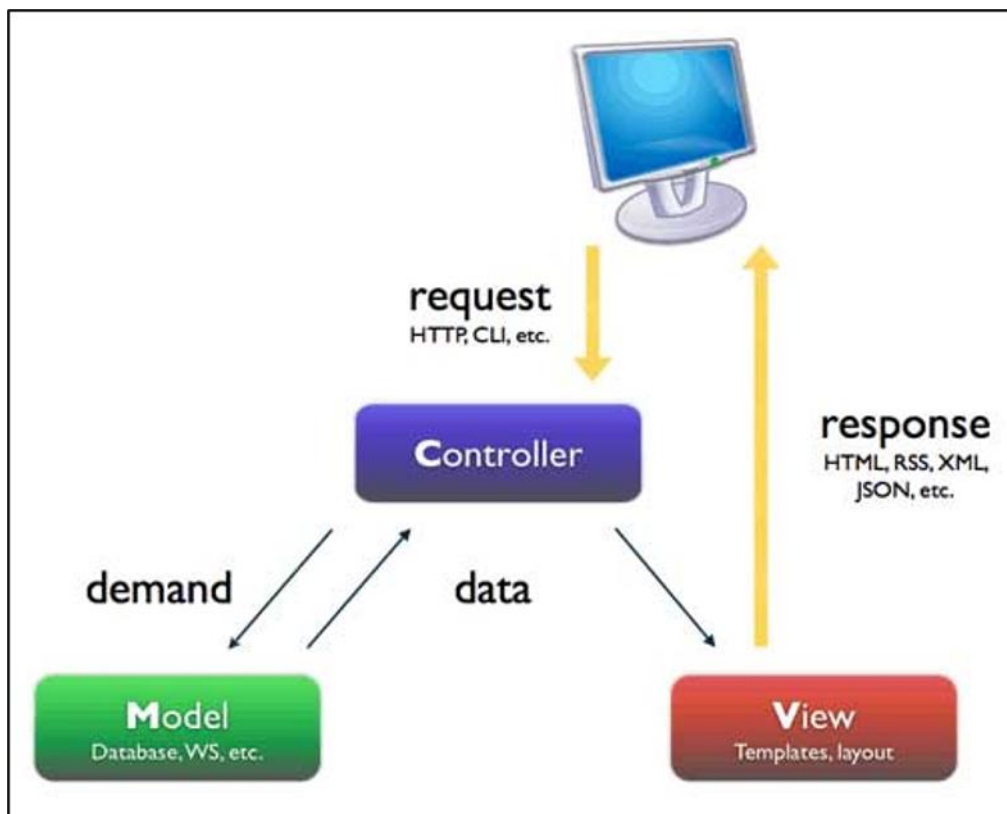
Para el desarrollo del Sistema Web de Control de Préstamos de la Biblioteca de Alejandría se utilizarán patrones que permitan organizar correctamente la comunicación entre la interfaz, la lógica del negocio y la base de datos. Estos patrones contribuirán a que la aplicación desarrollada con Java y Spring Boot mantenga una estructura más ordenada y facilite el trabajo de desarrollo y mantenimiento del sistema.

2.1.4.1. Patrón Modelo–Vista–Controlador (MVC)

El patrón Modelo–Vista–Controlador, conocido como MVC, es un patrón arquitectónico que divide una aplicación en tres componentes principales: el Modelo, la Vista y el Controlador. El Modelo se encarga de representar y gestionar los datos del sistema, la Vista corresponde a la interfaz con la que interactúa el usuario y el Controlador actúa como intermediario, recibiendo las acciones realizadas por el usuario y coordinando la respuesta del sistema.

Este patrón será utilizado en el Sistema Web de Control de Préstamos de la Biblioteca de Alejandría debido a que permite separar la interfaz gráfica de la lógica y gestión de los datos. Por ejemplo, las vistas desarrolladas con HTML y Thymeleaf permitirán mostrar la información al usuario, mientras que los controladores de Spring Boot procesarán las solicitudes realizadas y se comunicarán con las demás capas del sistema. Esta separación facilitará la organización del proyecto y permitirá realizar modificaciones en una parte del sistema sin afectar directamente a las demás.

Figura 15. Diagrama del patrón Modelo Vista Controlador



Fuente: A Software Architecture for Inventory Management System - ResearchGate (2012)

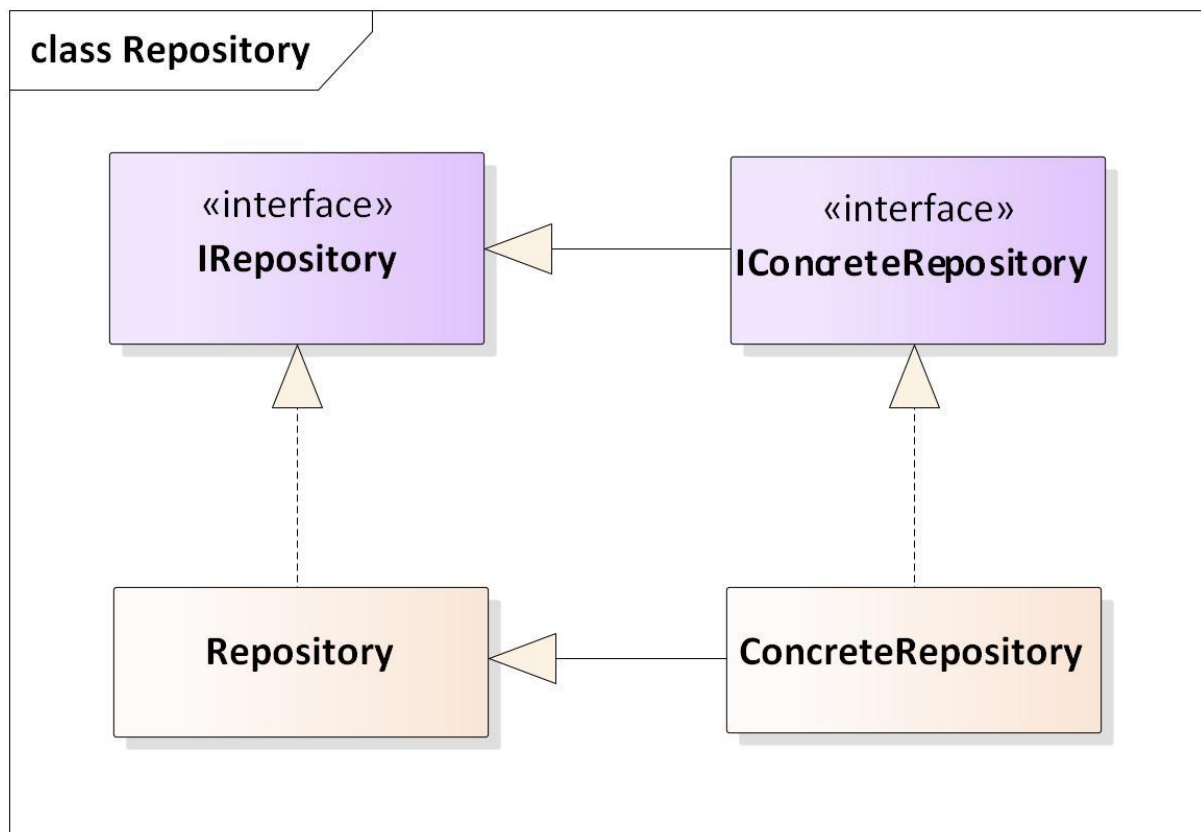
2.1.4.2. Patrón Repository

El patrón Repository permite separar las operaciones relacionadas con el acceso a los datos de la lógica principal de la aplicación. Su función es actuar como una capa intermediaria entre el sistema y la base de datos, concentrando las operaciones necesarias para consultar, registrar, actualizar o eliminar información. De esta

manera, los demás componentes del sistema no necesitan interactuar directamente con la base de datos.

En el Sistema Web de Control de Préstamos de la Biblioteca de Alejandría, este patrón permitirá organizar el acceso a la información almacenada en MySQL mediante repositorios relacionados con las diferentes entidades del sistema, como usuarios, libros y préstamos. Su implementación mediante Spring Boot facilitará la gestión de las operaciones de persistencia y permitirá mantener una mejor separación entre la lógica del negocio y el acceso a los datos, haciendo que el código sea más ordenado y fácil de mantener.

Figura 16. Diagrama del Patrón Repository



Fuente: Ofner, W. (2018).

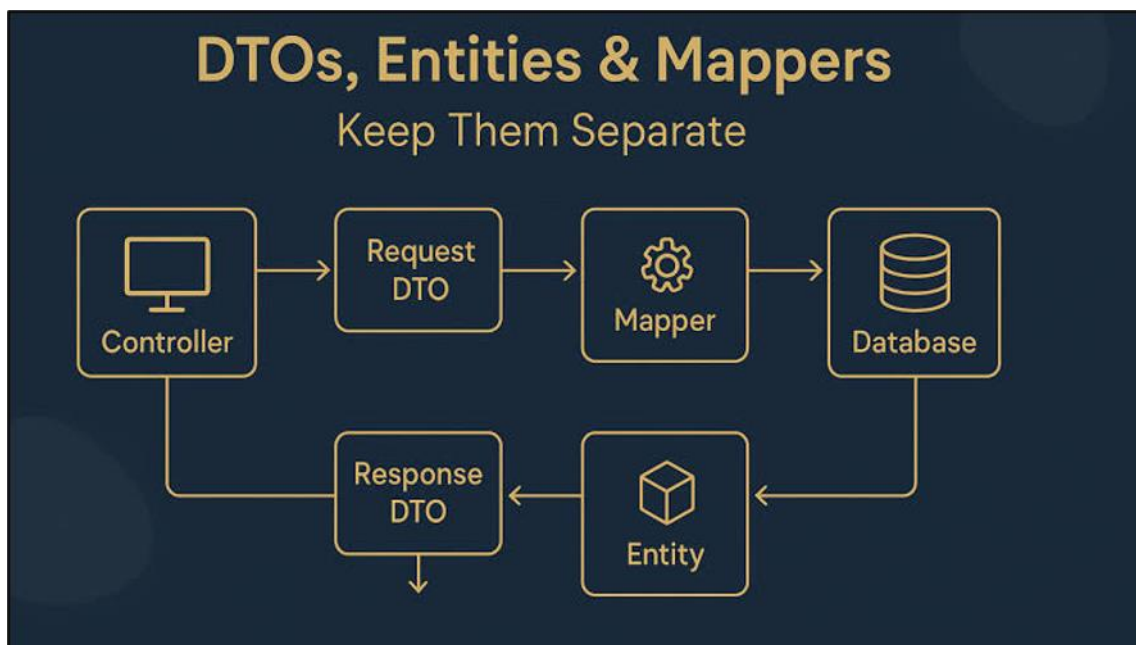
2.1.4.3. Patrón DTO (Data Transfer Object)

El patrón DTO, conocido como Data Transfer Object u Objeto de Transferencia de Datos, es utilizado para transportar información entre las diferentes capas de una

aplicación. En lugar de enviar directamente las entidades que representan los datos de la base de datos, se utilizan objetos específicos que contienen únicamente la información necesaria para una determinada operación o para mostrarla en la interfaz.

Este patrón será utilizado en el Sistema Web de Control de Préstamos de la Biblioteca de Alejandría para facilitar la transferencia de información entre los controladores, servicios y vistas del sistema. Por ejemplo, se podrán utilizar objetos DTO para enviar únicamente los datos necesarios de un usuario, libro o préstamo, evitando exponer información que no sea requerida. Esto permitirá mantener una mejor organización de los datos y facilitará el control de la información que circula entre las diferentes partes de la aplicación.

Figura 17. Diagrama del Patrón DTO



Fuente: Sandaruwan, D. (2025).

2.1.5 Lenguajes de programación

Para el desarrollo del Sistema Web de control de préstamo de libros de la "Biblioteca de Alejandría", se eligieron cuidadosamente los lenguajes de programación, frameworks y herramientas que mejor se adaptan a las necesidades del proyecto, considerando aspectos como rendimiento, facilidad de uso, mantenimiento y escalabilidad. La combinación de estas tecnologías permite estructurar una aplicación bajo estándares modernos de ingeniería de software, garantizando una arquitectura robusta tanto en el frontend como en el backend.

2.1.5.1. Lenguajes de programación

1. JavaScript

JavaScript se ha posicionado como el lenguaje predominante para dotar de dinamismo e interactividad a las interfaces web. En el contexto de la "Biblioteca de Alejandría", este lenguaje permite que el sistema responda de manera ágil a las acciones del usuario sin necesidad de recargar completamente la página.

- **Características Técnicas:** Su modelo de ejecución está basado en un bucle de eventos (*Event Loop*), lo que le permite manejar múltiples operaciones de manera asíncrona sin bloquear la interfaz gráfica. Esta cualidad es esencial para ejecutar peticiones al servidor (mediante AJAX o Fetch API) y actualizar dinámicamente el estado de los libros en el catálogo, permitiendo que el personal de la biblioteca verifique la disponibilidad de ejemplares sin necesidad de recargar toda la página.
- **Ventajas:** La integración nativa de JavaScript con el navegador posibilita una manipulación ágil del DOM (*Document Object Model*). Esto facilita la validación instantánea de los formularios (como los de registro de préstamos y devoluciones) y la creación de elementos interactivos que mejoran notablemente la experiencia del usuario, haciendo que el sistema se sienta más ágil y profesional.

Figura 18. Logo de JavaScript



Fuente: JavaScript.com (s.f.)

2. SQL (Structured Query Language)

SQL es el lenguaje estándar utilizado para comunicarse con bases de datos relacionales. En este proyecto, actúa como el vínculo fundamental entre la lógica de negocio del backend y el almacenamiento persistente de los datos.

- **Características Técnicas:** Se basa en el álgebra relacional para ejecutar operaciones de consulta, inserción, actualización y eliminación de datos. Permite la definición de estructuras complejas mediante el lenguaje de definición de datos (DDL) y la manipulación de registros a través del lenguaje de manipulación de datos (DML). Además, garantiza la integridad referencial mediante restricciones como claves primarias y foráneas, lo cual es crucial para mantener la coherencia entre las tablas de usuarios, libros y préstamos.
- **Ventajas:** Es un lenguaje declarativo y eficiente para manejar grandes volúmenes de información. Su estandarización asegura la escalabilidad del sistema y facilita la generación de reportes detallados mediante consultas complejas (*JOINS*), agrupaciones y filtros avanzados. Esto es vital para obtener información precisa sobre los préstamos, la disponibilidad de libros y las incidencias registradas en la biblioteca.

Figura 19. Logo de SQL



Fuente: Lenguajes de Programación (2025)

2.1.5.2. Frameworks e IDE

1. Spring Boot

Spring Boot es un framework que forma parte del ecosistema de Java y está diseñado para agilizar el desarrollo de aplicaciones empresariales. En este proyecto, actúa

como el núcleo del backend, orquestando la lógica de negocio, la seguridad y la interacción con la base de datos.

- **Características Técnicas:** Aprovecha los principios de Inyección de Dependencias (DI) y Programación Orientada a Aspectos (AOP) para mantener los componentes del sistema desacoplados y modularizados. Además, incorpora un servidor de aplicaciones embebido (Apache Tomcat), lo que convierte a la aplicación en un ejecutable autónomo (*self-contained*), simplificando su despliegue y ejecución.
- **Ventajas:** Spring Boot reduce drásticamente la configuración inicial mediante sus "Starters", que agrupan dependencias comunes. Ofrece un sistema de seguridad robusto a través de Spring Security, fundamental para gestionar el control de acceso por roles (Administrador, Encargado, Usuario). Asimismo, facilita la creación de endpoints REST que conectan de manera eficiente el sistema con la base de datos MySQL, gestionando las operaciones críticas como préstamos, devoluciones, reservas e incidencias.

Figura 20. Logo de Spring Boot



Fuente: Spring (s.f.)

2. Thymeleaf

Thymeleaf es un motor de plantillas Java del lado del servidor (*server-side*) que se utiliza para generar contenido HTML dinámico. Su propósito es transformar los datos provenientes del backend en interfaces web listas para ser visualizadas por el usuario.

- **Características Técnicas:** Su rasgo más distintivo es el concepto de *Plantillas Naturales*. Esto significa que los archivos HTML que utiliza Thymeleaf pueden ser abiertos y visualizados directamente en un navegador sin necesidad de estar en un servidor, lo que facilita la colaboración entre diseñadores y desarrolladores. Utiliza atributos especiales (como `th:each`, `th:if`) para inyectar lógica de control dentro del HTML, permitiendo, por ejemplo, listar dinámicamente los libros disponibles o mostrar el historial de préstamos.
- **Ventajas:** Su integración con Spring Boot es fluida y transparente, lo que permite una comunicación sencilla y segura entre los controladores y las vistas. Esto agiliza el desarrollo de interfaces dinámicas, como los formularios de gestión de préstamos, reservas y la generación de reportes mensuales.

Figura 21. Logo de Thymeleaf



Fuente: Thymeleaf (s.f.)

3. Bootstrap

Bootstrap es un framework de código abierto para el desarrollo frontend que combina CSS y JavaScript, permitiendo crear interfaces web responsivas y adaptativas a cualquier dispositivo.

- **Características Técnicas:** Se fundamenta en un sistema de rejilla (*Grid System*) de 12 columnas basado en Flexbox, que facilita la creación de diseños flexibles y adaptables. Incluye una biblioteca extensa de componentes

preestilizados (botones, tarjetas, tablas, barras de navegación, modales, etc.) que cumplen con estándares de accesibilidad y usabilidad.

- **Ventajas:** Asegura que el sistema de la "Biblioteca de Alejandría" se visualice correctamente en cualquier dispositivo, ya sea un ordenador de escritorio, una tablet o un teléfono móvil. Al proporcionar una base visual consistente y moderna, reduce significativamente la necesidad de escribir CSS personalizado, permitiendo que el equipo se concentre en la funcionalidad y la lógica de negocio.

Figura 22. Logo de Bootstrap



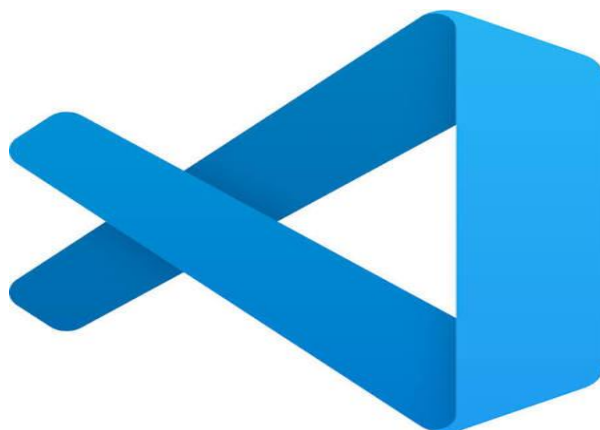
Fuente: Bootstrap (s.f.-a)

4. Visual Studio Code

Visual Studio Code es un editor de código fuente ligero, gratuito y altamente extensible, diseñado para optimizar el flujo de trabajo de los desarrolladores modernos.

- **Características Técnicas:** Aunque es un editor, a través de su arquitectura de extensiones puede equipararse a un IDE completo. Ofrece soporte nativo para Git, una terminal integrada y herramientas de depuración avanzadas que facilitan la identificación y corrección de errores en el código.
- **Ventajas:** Su tecnología IntelliSense proporciona autocompletado inteligente y sugerencias contextuales basadas en el tipo de variables y funciones, lo que minimiza los errores sintácticos y acelera la escritura de código limpio. Con las extensiones adecuadas (Java, Spring Boot, Thymeleaf, MySQL), se convierte en el entorno ideal para desarrollar todas las capas del proyecto.

Figura 23. Logo de Visual Studio Code



Fuente: Microsoft (s.f.-d)

2.1.6 Gestor de base de datos

MySQL

Para el almacenamiento y administración de la información del sistema de préstamos de la "**Biblioteca de Alejandría**", se ha optado por MySQL como sistema gestor de base de datos (SGBD). MySQL es el sistema de gestión de bases de datos relacionales (RDBMS) de código abierto más utilizado a nivel mundial. Basado en SQL, MySQL es un pilar fundamental en el desarrollo de aplicaciones web modernas, gracias a su alto rendimiento, fiabilidad comprobada y facilidad de implementación.

1. Descripción Técnica

MySQL opera bajo un modelo cliente-servidor y se caracteriza por su arquitectura de almacenamiento conectable, que permite a los administradores elegir el motor más adecuado para cada escenario. **InnoDB** es el motor predeterminado y el más recomendado, ya que ofrece soporte completo para transacciones ACID y claves foráneas. Es una solución multiplataforma que se integra de manera nativa con Java (a través de Spring Boot) y JavaScript.

2. Características Principales

Las funcionalidades de MySQL proporcionan las siguientes ventajas técnicas para el sistema de la "Biblioteca de Alejandría":

- **Código Abierto (Open Source):** Al ser gratuito bajo la licencia GPL, elimina cualquier barrera económica para el proyecto sin comprometer la potencia o la

calidad del software. Esto es particularmente valioso para una biblioteca que busca optimizar sus recursos financieros.

- **Rendimiento Optimizado:** Es ampliamente reconocido por su velocidad en operaciones de lectura, lo que garantiza que las consultas de disponibilidad de libros, el historial de préstamos y la generación de reportes mensuales se ejecuten de manera casi instantánea. Esto permite cumplir con el tiempo de respuesta máximo de 2 segundos establecido en los requerimientos no funcionales (RNF3).
- **Seguridad Sólida:** Incluye un sistema de privilegios de acceso basado en el host y cifrado de contraseñas, protegiendo la integridad y confidencialidad de la información. Esto se alinea directamente con el requerimiento de seguridad (RNF4).
- **Flexibilidad de Motores:** La utilización del motor InnoDB asegura el cumplimiento de las propiedades ACID, permitiendo el manejo seguro de transacciones. Esto es fundamental para operaciones críticas como el registro simultáneo de un préstamo y la actualización del estado del libro, evitando inconsistencias en el inventario.
- **Facilidad de Administración:** Dispone de herramientas potentes como MySQL Workbench, que permite el modelado visual de la base de datos, la ejecución de consultas y la gestión administrativa de forma gráfica e intuitiva, simplificando el mantenimiento a largo plazo.

Figura 24. Logo de MySQL



Fuente: Oracle (s.f.)

Tabla 26. Comparación técnica: MySQL vs. SQL Server vs. SQL Express

Característica	MySQL	SQL Server	SQL Express
Licencia	Código abierto	Propietario	Propietario
Costo	Gratis	Pago	Gratis con limitaciones
Código	Abierto	Cerrado	Cerrado
Motor	MyISAM e InnoDB	Motor propio	Motor propio
Plataformas	FreeBSD, Linux, OS X, Solaris, Windows	Linux, Windows	Linux, Windows
ACID	Depende del motor	Sí	Sí
Replicación	Master-Master / Master-Slave	Sí	No
Transacciones	Sí	Sí	Sí
Reportes	No	Sí	Sí
Limite memoria	No	No	Sí

Fuente: Guiadev (s.f.). MySQL vs SQL Server: Análisis y Comparación

3. Justificación de la elección

La decisión de emplear MySQL como Sistema Gestor de Base de Datos Relacional para la "Biblioteca de Alejandría" se fundamenta principalmente en su naturaleza de código abierto, lo que elimina por completo los costos de licenciamiento, y en su motor de almacenamiento InnoDB, que garantiza el cumplimiento riguroso de las propiedades ACID. Esto asegura la integridad, consistencia y aislamiento de los datos incluso en escenarios de acceso concurrente, previniendo problemas críticos como la asignación duplicada de un mismo libro a dos usuarios diferentes en la sala de lectura.

Adicionalmente, destaca por su excelente rendimiento en operaciones de lectura, lo que acelera las consultas del catálogo y la generación de reportes. Su integración con el stack tecnológico seleccionado (Java y Spring Boot) es nativa y fluida, facilitando el desarrollo y la conexión con las entidades del sistema (Usuarios, Libros, Préstamos, Reservas, Incidencias y Sanciones), asegurando así una solución robusta y confiable.

2.1.7 Herramientas adicionales

Para la gestión integral del proyecto, se han seleccionado diversas herramientas de software que facilitan la planificación, el modelado de procesos, el diseño de la experiencia de usuario y la documentación técnica. Estas herramientas no pertenecen a la categoría de lenguajes de programación ni de bases de datos, sino que actúan como soporte estratégico para asegurar la calidad y el orden en cada fase del desarrollo.

A. Pencil Project

Es una herramienta de código abierto diseñada para la creación de prototipos de interfaz de usuario (GUI) y wireframes.

- **Función:** Permite diseñar los bocetos iniciales de las pantallas del sistema de inventario, definiendo la ubicación de los botones, formularios y tablas antes de iniciar la fase de codificación.
- **Ventajas:** Ofrece una amplia gama de elementos predefinidos para aplicaciones web y de escritorio, lo que facilita la comunicación visual de la estructura de la interfaz con los interesados del proyecto.

Figura 25. Interfaz de Pencil Project



Fuente: Pencil Project (s.f.)

B. Bizagi Modeler

Es una aplicación de escritorio líder en el modelado de procesos de negocio utilizando el estándar BPMN (Business Process Model and Notation).

- **Función:** Se utiliza para diagramar el flujo de trabajo del negocio, como el proceso de registro de una venta o la recepción de mercadería, permitiendo identificar cada paso, actor y decisión dentro del sistema.
- **Ventajas:** Ayuda a comprender la lógica operativa de manera gráfica, asegurando que el software se alinee perfectamente con los procesos reales de la tienda.

Figura 26. Interfaz de Bizagi Modeler



Fuente: Bizagi (s.f.)

C. Lucidchart

Es una plataforma de diagramación basada en la nube que permite la creación colaborativa de esquemas técnicos y lógicos.

- **Función:** Se utiliza para elaborar los Diagramas Entidad-Relación (DER), empleando la notación Chen para el diseño conceptual y la notación Crow's Foot para el diseño lógico, así como para diagramas de arquitectura del sistema.
- **Ventajas:** Al ser una herramienta colaborativa, permite el acceso desde cualquier dispositivo y la exportación de diagramas en alta resolución para la documentación técnica.

Figura 27. Interfaz de Lucidchart



Fuente: Lucidchart (s.f.)

D. Canva

Es una herramienta de diseño gráfico en línea que utiliza una interfaz de arrastrar y soltar para la creación de contenido visual.

- **Función:** Se utiliza para el diseño de elementos gráficos auxiliares, la presentación de propuestas visuales de marca y la elaboración de manuales de usuario con una estética moderna y profesional.
- **Ventajas:** Facilita la creación de recursos visuales de alta calidad sin requerir conocimientos avanzados de diseño gráfico, manteniendo la identidad visual.

Figura 28. Interfaz de Canva



Fuente: Canva (s.f.)

E. Microsoft Word

Es el procesador de textos estándar de la industria, utilizado para la creación y gestión de documentos complejos.

- **Función:** Es la herramienta principal para la redacción de este informe de proyecto, incluyendo el marco teórico, los requerimientos del sistema y la documentación técnica final.
- **Ventajas:** Ofrece herramientas avanzadas de edición, revisión ortográfica, generación automática de índices y gestión de referencias bibliográficas bajo normas APA.

Figura 29. Interfaz de Microsoft Word



Fuente: Microsoft (s.f.-b)

F. Microsoft Excel

Es una aplicación de hojas de cálculo que permite realizar cálculos complejos, gestionar listas de datos y generar gráficos estadísticos.

- **Función:** En el contexto de este proyecto, se utiliza para la elaboración del diccionario de datos, el seguimiento de los requerimientos funcionales y el control de los tiempos de desarrollo en la fase de planificación.
- **Ventajas:** Su capacidad de organización en tablas y celdas lo hace ideal para estructurar metadatos y realizar comparativas técnicas de manera ordenada y precisa.

Figura 30. Interfaz de Microsoft Excel



Fuente: Microsoft (s.f.-a)

2.1.8 Diagrama de Clases

El Diagrama de Clases es un tipo de diagrama de estructura estática que describe la estructura de un sistema mostrando sus clases, atributos, operaciones (o métodos) y las relaciones entre los objetos. Es el pilar del diseño orientado a objetos, ya que permite visualizar la arquitectura del software antes de ser codificado, facilitando la detección de errores de diseño y asegurando que el sistema sea escalable y mantenible

A continuación, se explican los conceptos principales relacionados con el diagrama de clases:

A. Clase

La clase es la unidad básica del diagrama. Se puede definir como un "molde" o plantilla conceptual que define las características y comportamientos que compartirán todos los objetos de ese tipo. En el diagrama, se representa mediante un rectángulo dividido en tres secciones: el nombre de la clase (en la parte superior), sus atributos (en el centro) y sus métodos (en la inferior)

B. Objeto

Mientras que la clase es el molde, el objeto es la instancia concreta de esa clase. Si la clase es "Libro", un objeto sería "Cien años de soledad - Gabriel García Márquez". En los diagramas de clases, nos enfocamos en la plantilla (clase), pero el diseño se hace pensando en cómo esos objetos vivirán y morirán en la memoria del sistema durante la ejecución

C. Atributo y Método

Son los componentes internos que definen a la clase:

- **Atributo:** Los atributos representan los datos o el estado de la clase. Son las características que se desean capturar y almacenar para cada instancia. Actúan como variables dentro de la clase que guardan información específica (ej. título, autor, stock).
 - **Estructura en UML:** Se escriben siguiendo el formato: visibilidad nombreAtributo : tipoDato = valorInicial
 - **Tipos de datos:** Pueden ser tipos primitivos (int, double, boolean) o tipos complejos (objetos de otras clases como Categoria o Autor)
- **Método:** Representan el comportamiento o las capacidades funcionales de la clase. Son las acciones que los objetos pueden realizar o las peticiones que pueden atender. Un método se define por su nombre, sus parámetros y su tipo de retorno.
 - **Parámetros de un método:** Son los datos de entrada que un método requiere para procesar una acción. Sin estos valores, el método carecería de la información necesaria para ejecutar su lógica. Se declaran entre paréntesis indicando el nombre y el tipo: (parámetro: tipo).

Tipos de métodos según sus parámetros:

§ **Sin Parámetros:** El método no requiere datos externos para ejecutarse, ya que operan con la información que ya posee la clase o el sistema.

Ejemplo: Un método listarLibros(): Array procesa una operación interna y devuelve la lista completa de libros sin necesitar ningún valor de entrada.

§ **Con Parámetros:** El método requiere uno o más datos de entrada, para poder realizar su tarea de forma correcta. Estos datos se declaran entre paréntesis indicando el nombre y el tipo: (parámetro : tipo).

Ejemplo: buscarLibroPorIsbn(isbn): Libro recibe el código ISBN como parámetro para localizar un libro específico en el catálogo.

· **Tipos de Retorno de un método:** Al finalizar su ejecución, un método puede devolver un resultado al componente que lo invocó, o puede no devolver nada. Esto se define en su tipo de retorno

§ **Métodos con Retorno:** Devuelve un valor del tipo especificado (int, String, double, objetos, etc.)

Ejemplo: Un método calcularMulta(diasRetraso : int) : double procesa el número de días de retraso y devuelve un número decimal con el monto de la sanción correspondiente.

§ **Métodos void:** Se utilizan cuando el método debe ejecutar una acción o un cambio de estado interno, pero no necesita devolver ningún valor de vuelta. El término void (vacío) indica precisamente la ausencia de un valor de retorno.

Ejemplo: registrarDevolucion(prestamo : Prestamo) : void actualiza el estado del libro a "Disponible" y guarda los cambios en la base de datos, pero no genera un dato que deba ser procesado posteriormente por otra función.

· **Métodos Especiales:** Debido al principio de encapsulamiento, los atributos suelen ser privados. Para interactuar con ellos se utilizan:

- **Getters:** Métodos que retornan el valor de un atributo (ej. getTitulo(): String).
- **Setters:** Métodos de tipo void que reciben un parámetro para modificar el valor de un atributo de forma controlada (ej. setTitulo(nuevoTitulo: String): void).

D. Modificadores de Acceso: Público, Privado y Protegido

Los modificadores de acceso determinan el nivel de visibilidad de los atributos y métodos dentro del sistema:

- **Público (+):** Puede ser accedido desde cualquier parte del sistema.
- **Privado (-):** Solo puede ser accedido dentro de la misma clase. Es la base del encapsulamiento, protegiendo los datos internos.
- **Protegido (#):** Puede ser accedido dentro de la clase y por sus clases hijas (herencia), pero no desde el exterior.

Estos niveles ayudan a proteger la información y mantener el control sobre los datos.

E. Relaciones entre Clases

Una relación en un diagrama de clases es una conexión o vínculo entre dos o más clases que describe cómo interactúan entre sí dentro del sistema. Las relaciones permiten modelar la estructura del software y definir cómo se comunican los objetos para cumplir con la lógica de negocio. Existen diferentes tipos de relaciones, cada una con su propia semántica y notación:

- **Generalización (Herencia)**

Es una relación de tipo "es un". Permite que una clase hija herede atributos y métodos de una clase padre. Se representa con una línea sólida y una flecha de punta hueca dirigida al padre.

Ejemplo: Un "Administrador" es un "Usuario".

Notación: Una línea sólida con una flecha de punta triangular hueca que apunta hacia la clase padre.

- **Asociación**

Es una relación estructural que describe una conexión entre objetos de dos clases. Indica que los objetos interactúan entre sí pero no existe una dependencia de existencia. Se representa con una línea sólida que conecta ambas clases.

Ejemplo: Un "Usuario" realiza un "Préstamo".

- **Agregación**

Representa una relación de tipo "tiene un" o "todo/parte", donde las partes pueden existir independientemente del todo. Se representa con una línea sólida y un diamante hueco en el extremo del "todo".

Ejemplo: Una "Biblioteca" tiene "Libros". Si la biblioteca cierra, los libros siguen existiendo físicamente.

Notación: Una línea sólida con un diamante hueco en el extremo de la clase que representa el "todo".

- **Composición**

Es una forma más fuerte de agregación donde las partes no tienen sentido de existencia sin el todo (dependencia de vida). Se representa con un diamante relleno.

Ejemplo: Un préstamo tiene detalles de préstamo. Si el préstamo se elimina, el detalle no tiene razón de existir solo.

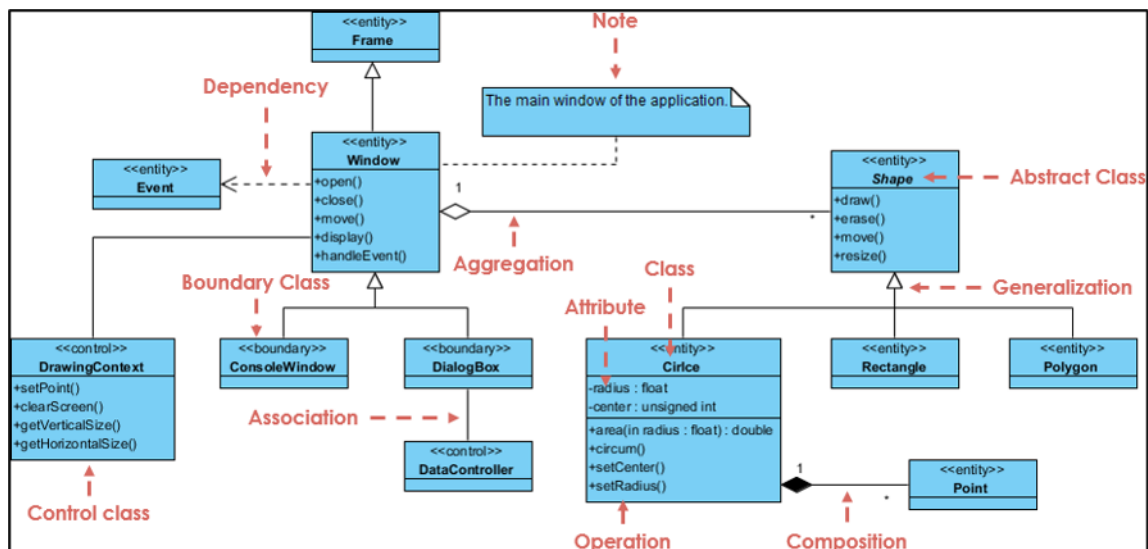
Notación: Una línea sólida con un diamante relleno en el extremo del contenedor.

- **Dependencia**

Es una relación mas debil que indica que una clase utiliza a otra de forma temporal. Se representa con una línea discontinua y una flecha abierta.

Ejemplo: Un "ReporteService" depende de "PrestamoRepository" para generar reportes.

Figura 31. Diagrama de Clases del sistema (fragmento ilustrativo)



Fuente: ArchiMetric (2025)

F. Relaciones Include y Extend

Es importante notar que los términos Include y Extend pertenecen originalmente al Diagrama de Casos de Uso, pero a menudo se discuten en el modelado para definir dependencias funcionales:

- Include (Inclusión): Indica que una función es obligatoria para que otra se complete. (Ej. Para "Realizar Venta", se incluye obligatoriamente "Validar Stock").
- Extend (Extensión): Indica que una función es opcional o solo ocurre bajo ciertas condiciones. (Ej. Al "Realizar Venta", se puede extender a "Aplicar Descuento de Temporada" solo si se cumple la condición).

G. Importancia del Diagrama de Clases

El diagrama de clases es esencial porque permite visualizar la estructura del sistema antes de su implementación, facilitando la organización del código, la detección de errores en el diseño y la comunicación entre los desarrolladores. Además, sirve como base para la programación orientada a objetos, asegurando que el sistema sea escalable, mantenible y bien estructurado.

2.2 Marco teórico de las fórmulas y/o sistema

El uso de indicadores dentro de un permite transformar datos abstractos en información accionable. Un indicador es una magnitud que expresa el comportamiento o desempeño de un proceso, cuya comparación con metas establecidas permite detectar desviaciones y tomar medidas correctivas.

A continuación, se describen los indicadores seleccionados para el sistema de acuerdo con su clasificación técnica:

A. Indicadores de Eficacia

La eficacia se centra exclusivamente en el cumplimiento de los objetivos. Mide el grado en que el sistema logra los resultados esperados, sin considerar necesariamente los recursos utilizados para ello.

- **Funcionamiento:** Relaciona el resultado alcanzado con la meta planificada (Resultado \Meta).
- **Utilidad:** Sirven para verificar si el sistema está cumpliendo con su propósito fundamental y si satisface los requisitos finales establecidos por los usuarios.
- **Ejemplo:** Porcentaje de pedidos entregados sin errores. Si el sistema tenía la meta de procesar y despachar 100 pedidos en el día y logró despachar los 100 de forma correcta, la eficacia fue del 100% (cumplió el objetivo a cabalidad, sin importar cuánto costó).

B. Indicadores de Eficiencia

A diferencia de la eficacia, la eficiencia mide la relación entre los resultados obtenidos y los recursos consumidos (tiempo, dinero, personal, capacidad de procesamiento).

- **Funcionamiento:** Se enfocan en la optimización. Un proceso es eficiente si logra el objetivo utilizando la mínima cantidad de recursos o si maximiza los resultados con un recurso fijo.
- **Utilidad:** Son críticos para evaluar la rentabilidad operativa y la productividad técnica del sistema.
- **Ejemplo:** Consumo de memoria RAM por hilo de ejecución. Si la versión anterior del sistema consumía 512 MB de RAM para procesar 50 usuarios

simultáneos, y tras una optimización de código ahora solo consume 256 MB para los mismos 50 usuarios, el sistema se volvió el doble de eficiente.

C. Indicadores de Calidad

Miden el nivel de conformidad de un producto o servicio con respecto a los estándares predefinidos. Se enfocan en la ausencia de errores y la satisfacción de las especificaciones.

- **Funcionamiento:** Generalmente se expresan como una tasa de error o porcentaje de éxito (ej. Número de errores \ Total de registros).
- **Utilidad:** Permiten monitorear la fiabilidad de los datos, la precisión de las operaciones y la integridad de la información procesada.
- **Ejemplo:** Tasa de rebote o tasa de error de software (Bugs). Si durante una jornada se realizan 1,000 transacciones bancarias en la plataforma y 5 de ellas fallan o arrojan un error de código, la tasa de error de calidad es del 0.5%.

D. Indicadores de Resultado

Miden el impacto final o el beneficio obtenido tras la ejecución de una serie de procesos. Se enfocan en la salida (output) del sistema y su efecto en la organización.

- **Funcionamiento:** Evalúan la situación final después de la intervención del sistema en comparación con una línea base previa.
- **Utilidad:** Proporcionan una visión de "alto nivel" sobre el éxito global de la implementación tecnológica.
- **Ejemplo:** Reducción del ausentismo estudiantil o incremento en ventas. Si tras implementar un sistema de alertas académicas automatizadas, el abandono de los estudiantes disminuye en un 12% comparado con el ciclo anterior. Mide el impacto final del software en el mundo real.

E. Indicadores de Proceso

Se enfocan en el monitoreo de las actividades intermedias. No evalúan el resultado final, sino cómo se están ejecutando las tareas paso a paso.

- **Funcionamiento:** Miden variables como el flujo de trabajo, la carga de trabajo por etapa y el cumplimiento de los procedimientos internos.
- **Utilidad:** Son preventivos; permiten detectar desviaciones antes de que el proceso termine, facilitando correcciones inmediatas.

- **Ejemplo:** Cuello de botella en la fase de validación. Monitorear cuántas solicitudes están actualmente en estado "Espera de aprobación por el supervisor". Si hay 80 solicitudes estancadas en ese paso intermedio, el indicador avisa que el proceso interno está fallando ahí antes de que afecte al resultado final.

F. Indicadores de Insumo

Miden la velocidad y el volumen en que se gastan o agotan los recursos asignados durante la operación activa del sistema.

- **Funcionamiento:** Registran de manera continua la tasa de gasto o uso de un recurso financiero, tecnológico o material a lo largo del tiempo.
- **Utilidad:** Ayudan a prevenir el desabastecimiento de insumos, controlar los costos operativos en tiempo real y evitar que el sistema colapse por falta de recursos a mitad del proceso.
- **Ejemplo:** Presupuesto asignado para licencias o servidores. Contar con un presupuesto inicial de \$5,000 USD o disponer de 4 desarrolladores senior para iniciar la fase de maquetación del sistema. Mide lo que necesitas antes de arrancar.

G. Indicadores de Economía

Miden la capacidad del sistema o del proyecto para movilizar, administrar y optimizar los recursos financieros de manera oportuna y al menor costo posible. A diferencia de la eficiencia (que ve la relación recurso/resultado), la economía se enfoca estrictamente en el ahorro y el costo de los recursos adquiridos.

- **Funcionamiento:** Comparan el costo real invertido en adquirir los insumos o mantener el sistema contra el presupuesto estimado o los costos del mercado.
- **Utilidad:** Permiten evaluar la sostenibilidad financiera del proyecto, asegurando que no existan sobrecostos en licencias, servidores o mano de obra, y que se respete el presupuesto asignado.
- **Ejemplo:** Desviación presupuestaria del proyecto. Si el presupuesto estimado para el desarrollo del módulo de pagos era de \$3,000 USD y el costo real final fue de \$2,800 USD, el indicador muestra un ahorro económico del 6.6%.

H. Indicadores de Producto

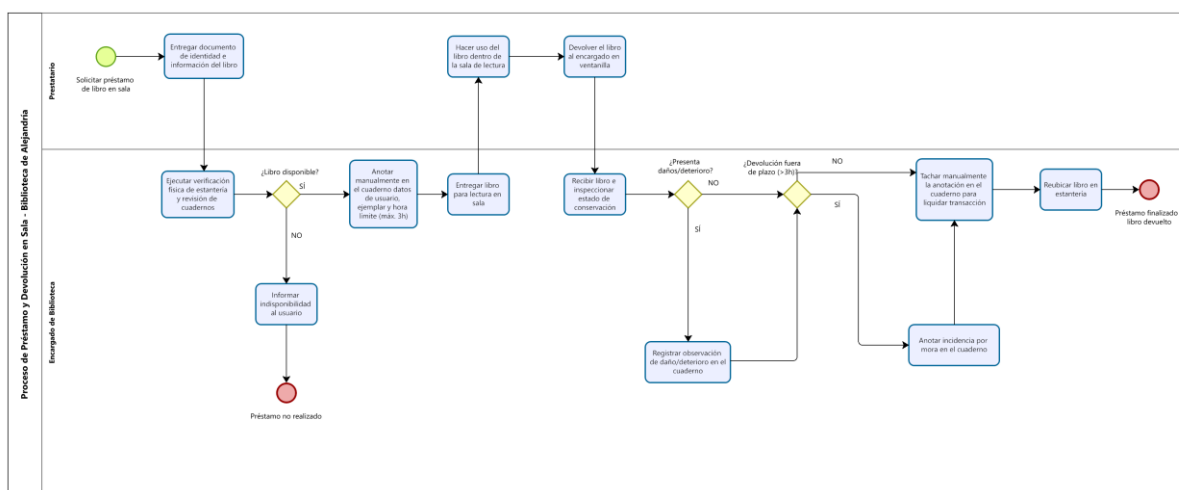
Miden la cantidad y las características de los bienes o servicios tangibles que el sistema genera directamente como resultado de sus operaciones. Se enfocan en el output directo y cuantificable de la plataforma.

- **Funcionamiento:** Contabilizan de forma directa las unidades producidas, módulos desarrollados, reportes generados o transacciones completadas con éxito en un periodo de tiempo.
- **Utilidad:** Sirven para comprobar el volumen de entregables que el sistema es capaz de proveer a los usuarios finales y asegurar que la "fábrica" del sistema funcione al ritmo planeado.
- **Ejemplo:** Cantidad de módulos de software desplegados en producción. Si el plan de trabajo de un sprint de desarrollo exigía entregar 4 nuevas funcionalidades (ej. carrito de compras, login, historial y perfil) y se subieron las 4 a la plataforma, el indicador de producto registra un cumplimiento del 100% en entregables.

CAPITULO 3 - DESARROLLO DE LA SOLUCION

3.1. Modelo de proceso de negocios (diagrama As-Is)

Figura 32. Modelo del Proceso de Negocio



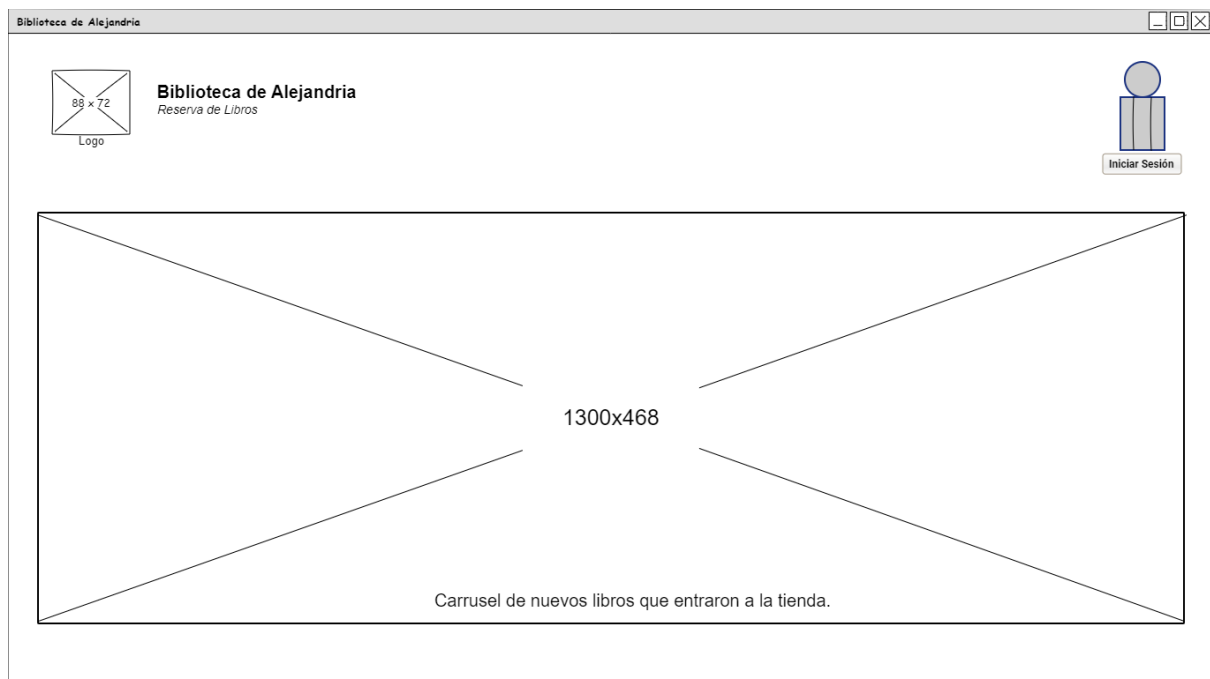
Fuente: elaboración propia

3.2. Prototipo interactivo

A continuación, se presentan los prototipos de las principales pantallas del Sistema Web de Control de Préstamos de la “Biblioteca de Alejandría”, elaborados como bocetos de baja fidelidad (wireframes) a partir de los requerimientos funcionales definidos en el apartado 1.4 del proyecto. Estos prototipos representan la distribución preliminar de los formularios, tablas y paneles que se necesitan implementar para resolver el problema planteado, y servirán de referencia para el desarrollo del entorno visual definitivo del sistema.

Pantalla principal y de bienvenida (Recibidor): Muestra el acceso público al sistema de la Biblioteca de Alejandría. Cuenta con un encabezado con el logotipo institucional, un botón directo para iniciar sesión en la esquina superior derecha y un carrusel principal destacado destinado a presentar las novedades bibliográficas o los libros incorporados recientemente al catálogo.

Figura 33. Pantalla principal y de bienvenida (Recibidor)



Fuente: elaboración pública

Pantalla de inicio de sesión de usuarios (Login): Permite la autenticación de los alumnos y lectores registrados mediante el ingreso de su nombre de usuario y contraseña. Incluye opciones para recuperar la contraseña olvidada, un acceso directo para registrarse como nuevo usuario e integra un botón de acceso administrativo (ADMIN) en el área inferior.

Figura 34. Pantalla de inicio de sesión de usuarios (Login)



Biblioteca de Alejandria - Inicio Sesión

Logo

Biblioteca de Alejandria
Reserva de Libros

INICIO DE SESION

Ingresa tu nombre de Usuario

Ingresa tu contraseña

[Olvidaste tu Contraseña?](#)

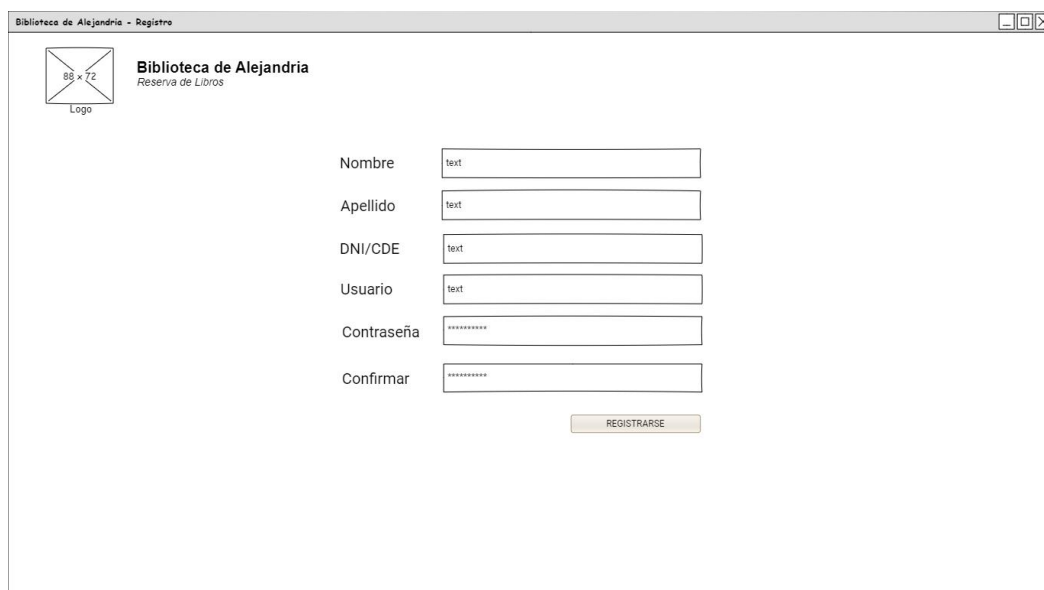
REGISTRARSE INGRESAR

ADMIN

Fuente: elaboración pública

Pantalla de registro de nuevos usuarios: Permite a los nuevos lectores darse de alta en la plataforma completando un formulario con sus datos básicos: nombre, apellido, nombre de usuario deseado, contraseña y confirmación de la misma, dando cumplimiento a la creación de perfiles en el sistema.

Figura 35. Pantalla de registro de nuevos usuarios



Biblioteca de Alejandria - Registro

Logo

Biblioteca de Alejandria
Reserva de Libros

Nombre

Apellido

DNI/CDE

Usuario

Contraseña

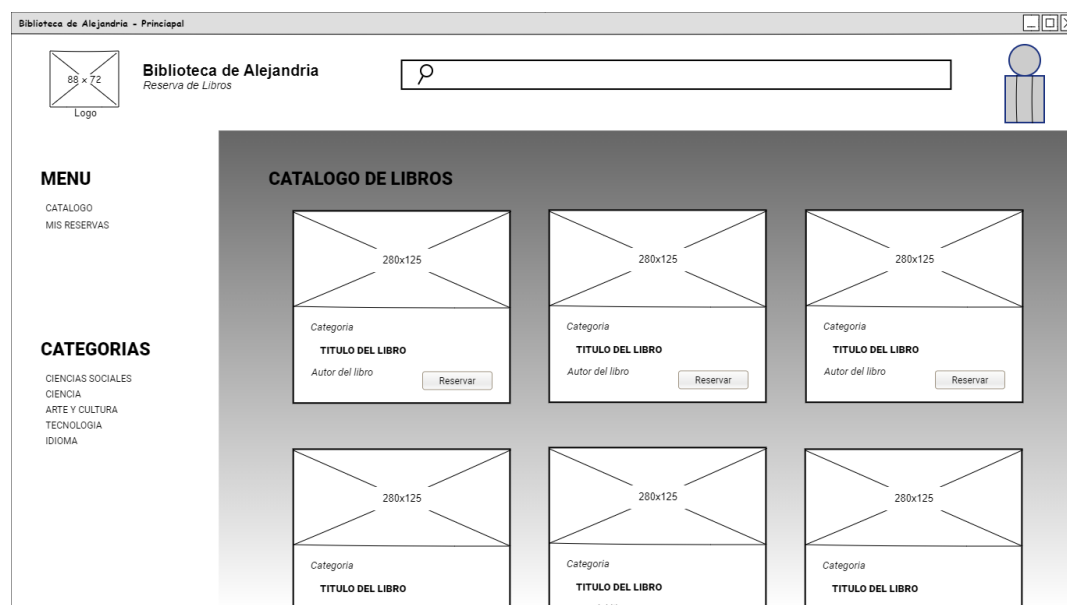
Confirmar

REGISTRARSE

Fuente: elaboración pública

Pantalla de catálogo bibliográfico: Presenta la oferta de libros organizada en una cuadrícula de tarjetas que muestran la portada, categoría, título, autor y un botón de acción rápida para Reservar. Dispone de una barra de búsqueda superior en tiempo real y un menú lateral desplegable que clasifica el material por áreas temáticas (Ciencias Sociales, Ciencia, Arte y Cultura, Tecnología e Idioma).

Figura 36. Pantalla de catálogo bibliográfico



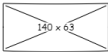
Fuente: elaboración pública

Pantalla de configuración y confirmación de reserva: Ventana modal que se despliega tras seleccionar un libro del catálogo. Permite al usuario verificar el título y autor seleccionados, así como elegir la fecha y el horario específico en el que acudirá a la biblioteca a recoger el ejemplar antes de confirmar la reserva.

Figura 37. Pantalla de configuración y confirmación de reserva

Biblioteca de Alejandria - Reservas

CONFIRMAR RESERVA

 **TITULO DEL LIBRO**
Autor del libro

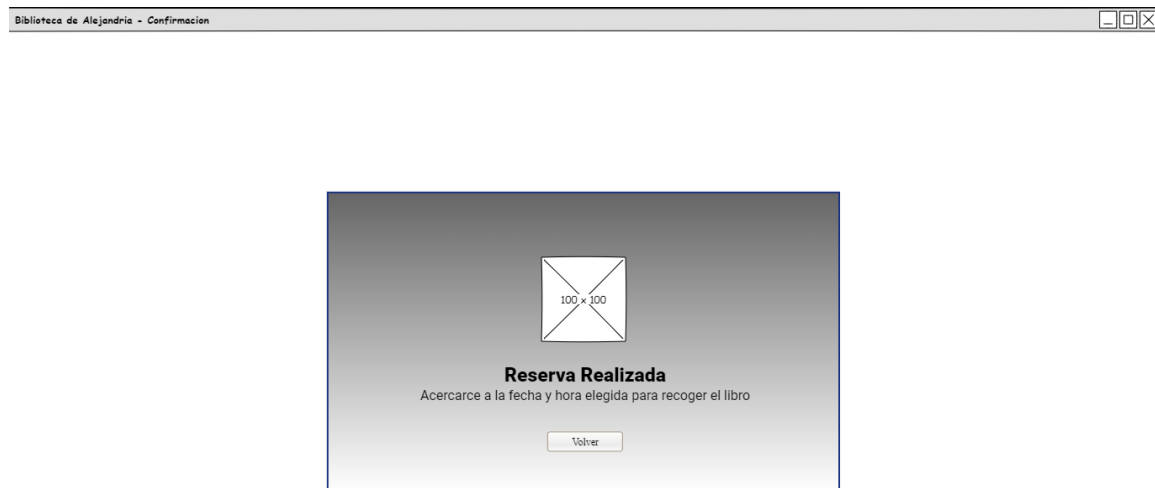
Selecciona la fecha de recojo:

Selecciona la hora de recojo:

Fuente: elaboración pública

Pantalla de confirmación de reserva realizada: Muestra un mensaje explícito de éxito confirmando que la transacción se registró correctamente en el sistema. Le indica al usuario la instrucción de acercarse en la fecha y hora seleccionadas para el recojo del material en sala e incluye un botón para volver al menú principal.

Figura 38. Pantalla de confirmación de reserva realizada



Fuente: elaboración pública

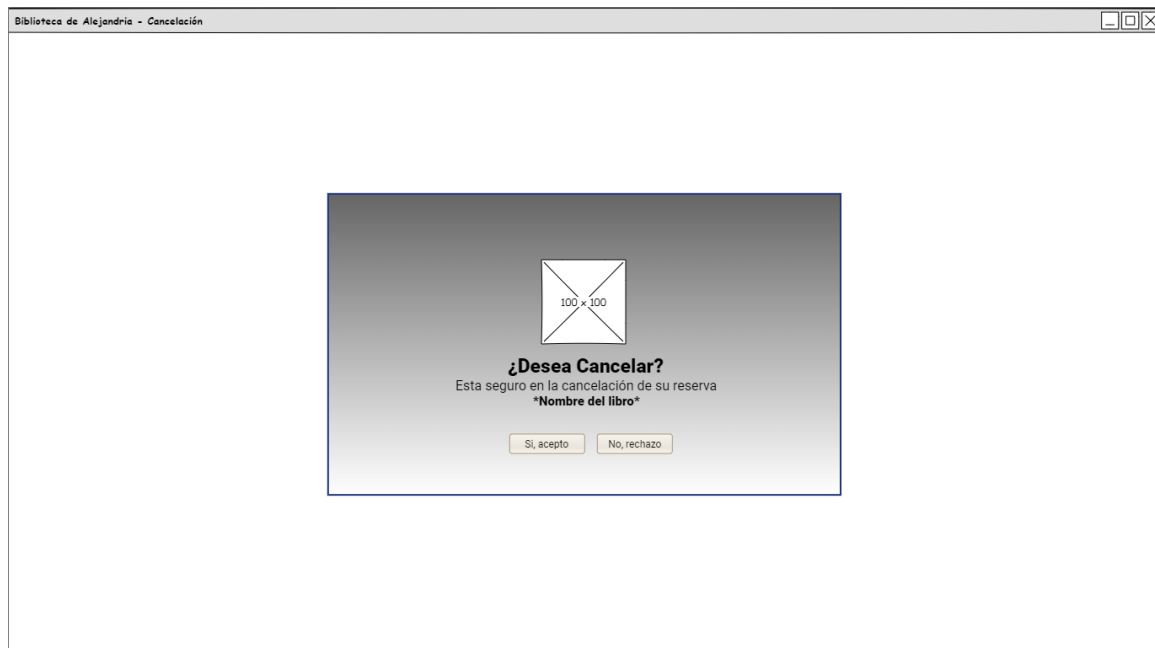
Pantalla de consulta de reservas del usuario (Mis Reservas): Muestra el panel personal donde el alumno puede consultar de manera organizada el listado completo de los libros que tiene actualmente reservados o prestados, permitiendo llevar un seguimiento de sus solicitudes activas.

Figura 39. Pantalla de consulta de reservas del usuario (Mis Reservas)

Fuente: elaboración pública

Pantalla de confirmación de cancelación de reserva (Cancelación): Muestra un cuadro de diálogo modal o de alerta donde se le solicita al usuario confirmar su decisión de anular una reserva activa. La interfaz presenta un icono o marcador de posición visual, un mensaje de advertencia ("¿Desea Cancelar?"), el texto explicativo sobre la acción a realizar junto con el nombre del libro seleccionado, y dos botones de opción o respuesta ("Sí, acepto" y "No, rechazo") para proceder o abortar la solicitud.

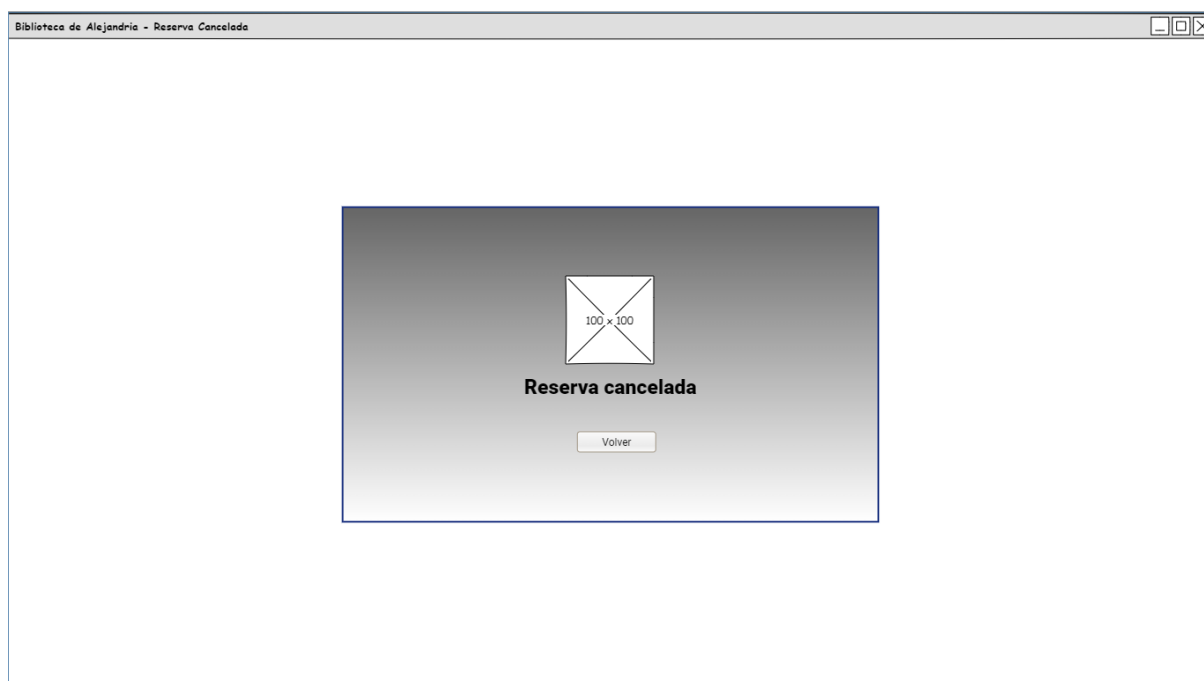
Figura 40. Pantalla de confirmación de cancelación de reserva:



Fuente: elaboración pública

Pantalla de notificación de reserva cancelada (Reserva Cancelada): Muestra un cuadro de diálogo informativo o de aviso donde se le comunica al alumno que el proceso de anulación de su solicitud se ha completado con éxito. La interfaz presenta un icono o marcador de posición visual, un mensaje principal que confirma el estado ("Reserva cancelada") y un botón único de acción ("Volver") para retornar al panel principal o al listado de reservas activas.

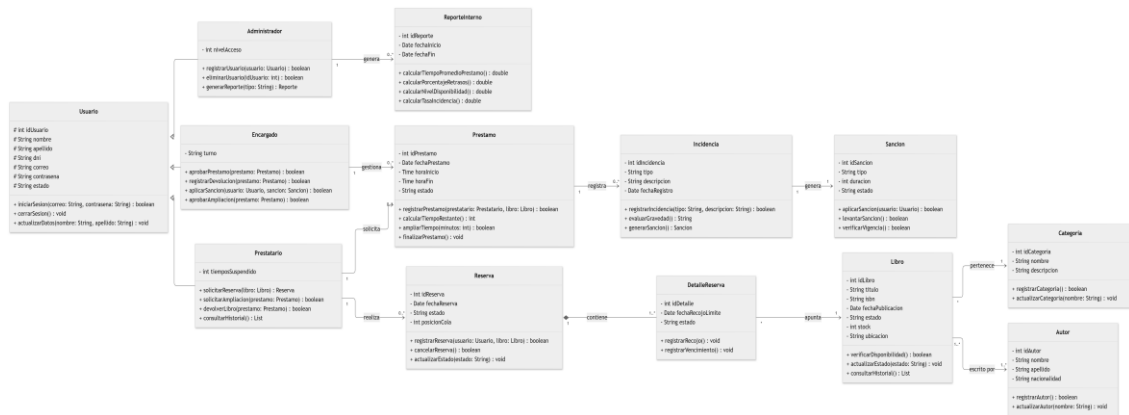
Figura 41. Pantalla de notificación de reserva cancelada



Fuente: elaboración pública

3.3 Diagrama de clases

Figura 42. Diagrama de clase Biblioteca de Alejandra

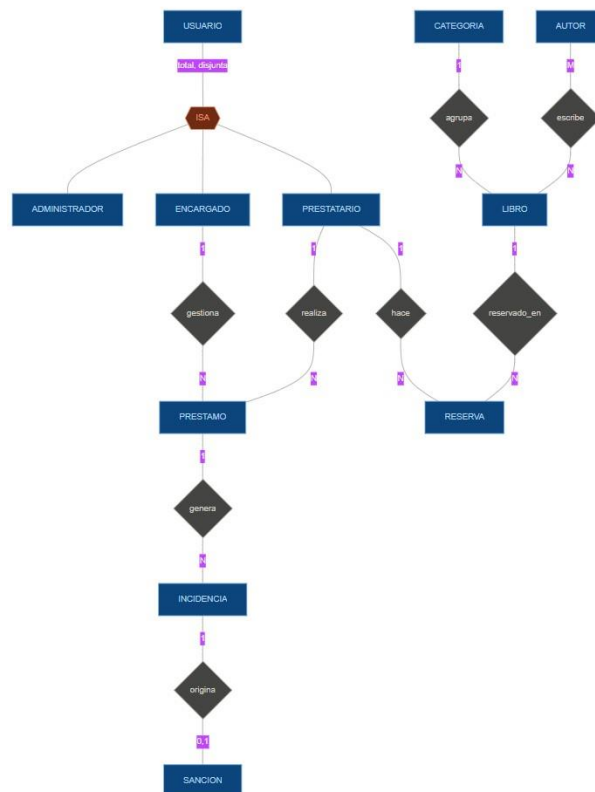


Fuente: Elaboración propia

3.4 Modelamiento de base de datos

3.4.1. Modelo Entidad-Relación

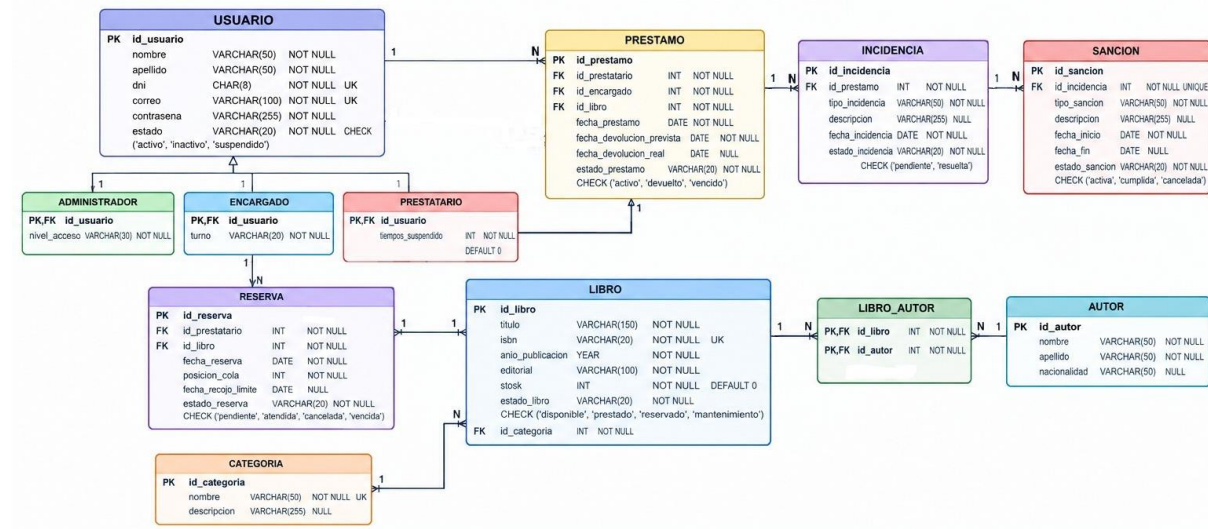
Figura 43. Modelo Conceptual de la base de datos de Biblioteca de Alejandra



Fuente: Elaboración propia

3.4.2. Modelo Lógico

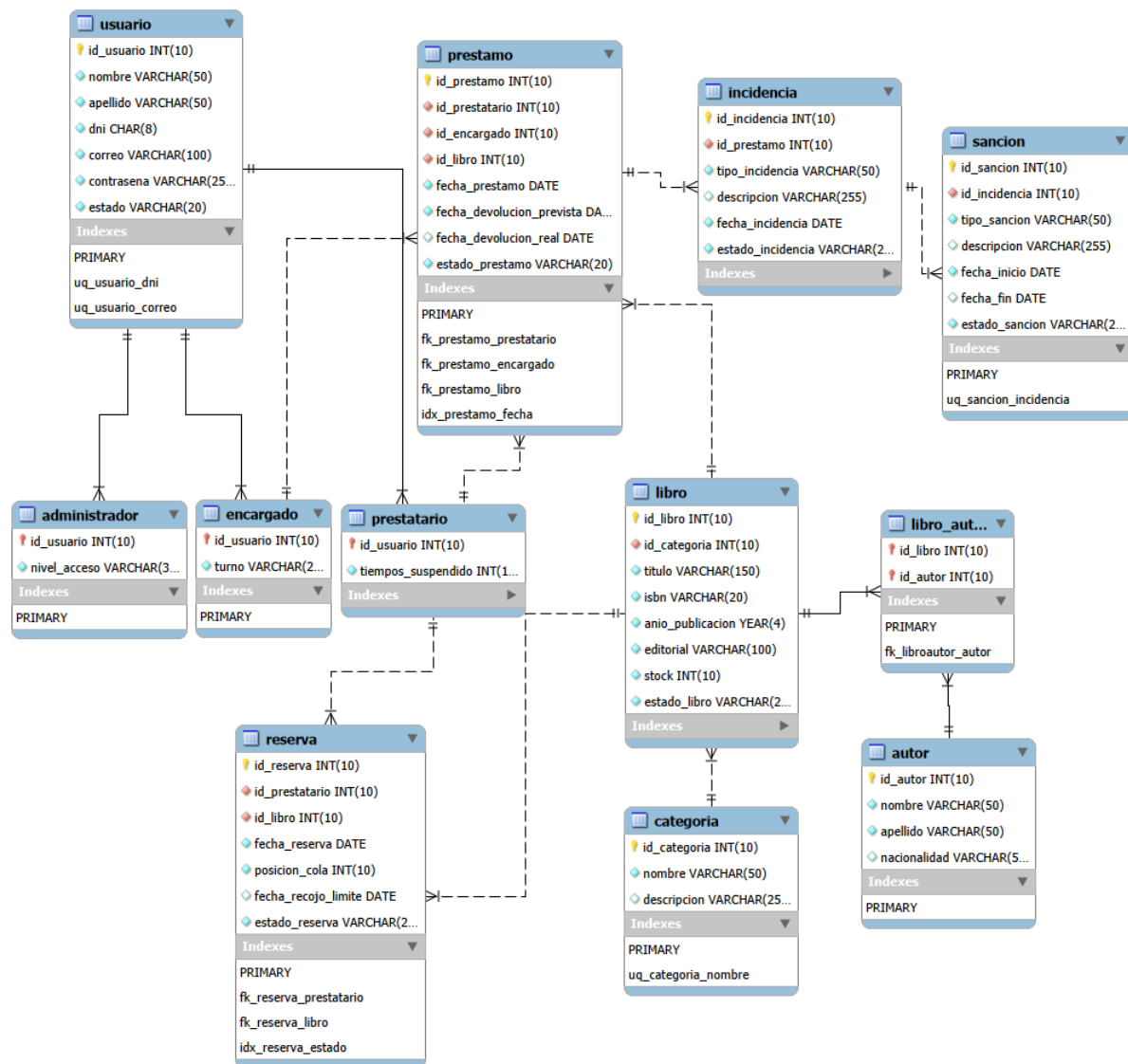
Figura 44. Modelo Lógico de la base de datos de Biblioteca de Alejandría



Fuente: Elaboración propia

3.4.3. Modelo Físico

Figura 45. Modelo Físico de la base de datos de Biblioteca de Alejandría



Fuente: Elaboración propia

3.4.4. Diccionario de datos

Tabla 27. Usuario

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
-------	---------	--------------	----------	-------------	-------------

PK	id_usuario	INT	11	PRIMARY KEY, AUTO_INCREMENT	Identificador único del usuario
	nombre	VARCHAR	50	NOT NULL	Nombre del usuario
	apellido	VARCHAR	50	NOT NULL	Apellido del usuario
	dni	CHAR	8	NOT NULL, UNIQUE	Documento Nacional de Identidad
	correo	VARCHAR	100	NOT NULL, UNIQUE	Correo electrónico
	contrasena	VARCHAR	255	NOT NULL	Contraseña cifrada (BCrypt)
	estado	VARCHAR	20	NOT NULL, CHECK	Estado: 'activo', 'inactivo', 'suspendido'

Tabla 28. Administrador

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
PK/FK	id_usuario	INT	11	PRIMARY KEY, FOREIGN KEY	Referencia al usuario (herencia)

	nivel_acceso	VARCHAR	30	NOT NULL	Nivel de acceso al administrador
--	--------------	---------	----	----------	----------------------------------

Tabla 29. Encargado

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
PK/FK	id_usuario	INT	11	PRIMARY KEY, FOREIGN KEY	Referencia al usuario (herencia)
	turno	VARCHAR	20	NOT NULL, DEFAULT 0	Turno del encargado (mañana/tarde)

Tabla 30. Prestatario

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
PK/FK	id_usuario	INT	11	PRIMARY KEY, FOREIGN KEY	Referencia al usuario (herencia)
	tiempos_suspendido	INT	11	NOT NULL, DEFAULT 0	Turno del encargado (mañana/tarde)

Tabla 31. Categoría

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
-------	---------	--------------	----------	-------------	-------------

PK	id_categoria	INT	11	PRIMARY KEY, AUTO_INCREMENT	Identificador único de la categoria
	nombre	VARCHAR	50	NOT NULL, UNIQUE	Nombre de la categoria
	descripcion	VARCHAR	255	NOT NULL	Descripcion de la categoria

Tabla 32. Autor

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
PK	id_autor	INT	11	PRIMARY KEY, AUTO_INCREMENT	Identificador único del autor
	nombre	VARCHAR	50	NOT NULL	Nombre del autor
	apellido	VARCHAR	50	NOT NULL	Apellido del autor
	nacionalidad	VARCHAR	50	NULL	Nacionalidad del autor

Tabla 33. Libro

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
PK	id_libro	INT	11	PRIMARY KEY, AUTO_INCREMENT	Identificador único del libro

FK	Id_categoria	INT	11	FOREIGN KEY, NOT NULL	Referencia a la categoria
	titulo	VARCHAR	150	NOT NULL	Título del libro
	isbn	VARCHAR	20	NOT NULL, UNIQUE	Código ISBN
	Anio_publicacion	YEAR	4	NOT NULL	Año de la publicación
	editorial	VARCHAR	100	NOT NULL	Editorial
	stock	INT	11	NOT NULL, DEFAULT 0	Cantidad disponible
	estado_libro	VARCHAR	20	NOT NULL, CHECK	Estado: 'disponible', 'prestado', 'reservado', 'mantenimiento'

Tabla 34. Libro_autor

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
PK/FK	id_libro	INT	11	PRIMARY KEY, FOREIGN KEY	Referencia al libro
PK/FK	id_autor	INT	11	PRIMARY KEY, FOREIGN KEY	Referencia al autor

Tabla 35: Préstamo

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
PK	id_prestamo	INT	11	PRIMARY KEY, AUTO_INCREMENT	Identificador único del préstamo
FK	id_prestatario	INT	11	FOREIGN KEY, NOT NULL	Usuario que solicita el préstamo
FK	id_encargado	INT	11	FOREIGN KEY, NOT NULL	Encargado que gestiona el préstamo
FK	id_libro	INT	11	FOREIGN KEY, NOT NULL	Libro prestado
	fecha_prestamo	DATE	-	NOT NULL	Fecha del préstamo
	fecha_devolucion_prevista	DATE	-	NOT NULL	Fecha prevista de devolución
	fecha_devolucion_real	DATE	-	NULL	Fecha real de devolución

	estado_prestamo	VARCHAR	20	NOT NULL, CHECK	Estado: 'activo', 'devuelto', 'vencido'
--	-----------------	---------	----	--------------------	--

Tabla 36. Incidencia

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
PK	id_incidencia	INT	11	PRIMARY KEY, AUTO_INCREMENT	Identificador único de la incidencia
FK	id_incidencia	INT	11	FOREIGN KEY, NOT NULL	Prestamo asociado
	tipo_incidencia	VARCHAR	50	NOT NULL	Tipo: 'retraso', 'daño', 'pérdida'
	descripcion	VARCHAR	255	NOT NULL	Descripción de la incidencia
	fecha_incidencia	DATE	-	NOT NULL	Fecha de registro
	estado_incidencia	VARCHAR	20	NOT NULL, CHECK	Estado: 'pendiente', 'resuelta'

Tabla 37. Sanción

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
PK	id_sancion	INT	11	PRIMARY KEY, AUTO_INCREMENT	Identificador único de la sancion
FK	id_incidencia	INT	11	FOREIGN KEY, NOT NULL, UNIQUE	Incendencia que origina la sanción
	tipo_sancion	VARCHAR	50	NOT NULL	Tipo: 'advertencia', 'suspensión', 'multa'
	descripcion	VARCHAR	255	NULL	Descripción de la sanción
	fecha_inicio	DATE	-	NOT NULL	Fecha de inicio
	fecha_fin	DATE	-	NULL	Fecha de fin
	estado_sancion	VARCHAR	20	NOT NULL, CHECK	Estado: 'activa', 'cumplida', 'cancelada'

Tabla 38. Reserva

Clave	Columna	Tipo de dato	Longitud	Restricción	Descripción
-------	---------	--------------	----------	-------------	-------------

PK	id_reserva	INT	11	PRIMARY KEY, AUTO_INCREMENT	Identificador único de la reserva
FK	id_prestatario	INT	11	FOREIGN KEY, NOT NULL	Usuario que reserva
FK	id_libro	INT	11	FOREIGN KEY, NOT NULL	Libro reservado
	Fecha_reserva	DATE	-	NOT NULL	Fecha de la reserva
	posicion_cola	INT	11	NOT NULL	Posición en la cola de espera
	fecha_recojo_limite	DATE	-	NOT NULL	Fecha límite para escoger
	estado_reserva	VARCHAR	20	NOT NULL, CHECK	Estado: 'pendiente', 'atendida', 'cancelada', 'vencida'

3.5 Entorno visual de los formularios

BIBLIOGRAFÍA

Action. (s.f.). Diccionario de datos: definición y conceptos clave.
<https://www.action.com/es/glossary/data-dictionary/>

Asana. (2026). Qué es Scrum: marco ágil, sprints, roles y artefactos.
<https://asana.com/es/resources/what-is-scrum>

Atlassian. (s.f.-a). ¿Qué es Scrum? Desglose del marco de trabajo ágil.
<https://www.atlassian.com/es/agile/scrum/>

Atlassian. (s.f.-b). Scrum vs Agile: diferencias y ventajas.
<https://www.atlassian.com/es/agile/scrum/agile-vs-scrum>

Bagui, S. (2009). Mapping generalizations and specializations and categories to relational databases. In V. E. Ferragine, J. H. Doorn, & L. C. Rivero (Eds.), Handbook of research on innovations in database technologies and applications: Current and future trends (pp. 1–11). IGI Global. <https://doi.org/10.4018/978-1-60566-242-8.ch001>

Bizagi. (s.f.). Bizagi Modeler. <https://www.bizagi.com/es/plataforma/modeler>

Bootstrap. (s.f.-a). Bootstrap. <https://getbootstrap.com/>

Bootstrap. (s.f.-b). Documentación de Bootstrap.
<https://getbootstrap.com/docs/5.0/getting-started/introduction/>

Canva. (s.f.). Canva. https://www.canva.com/es_419/

Chira Roque, M. (2019). Desarrollo de un sistema informático para la gestión bibliotecaria. Universidad Católica Los Ángeles de Chimbote. <https://repositorio.uladech.edu.pe/handle/20.500.13032/11138>

Codervent. (s.f.). Codervent UI Blocks — shadcn/ui Components for React & Next.js. Recuperado el 9 de septiembre de 2026, de <https://codervent.com/templates/>

CodexWorld. (2022, 13 de noviembre). PHP Social Login System. <https://www.codexworld.com/scripts/php-social-login-system/>

DoneTonic. (s.f.). Los 5 eventos en Scrum claves para el desarrollo de producto. <https://donetonic.com/es/que-son-los-sprints-en-scrum/>

Gallardo Vásquez, J. (2022). Diseño de un sistema web con software libre para la gestión de biblioteca. Universidad Católica Los Ángeles de Chimbote. <https://repositorio.uladech.edu.pe/handle/20.500.13032/43430>

GeeksforGeeks. (s.f.). UML Class Diagram. <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/>

IBM. (s.f.). ¿Qué es SQL (lenguaje de consulta estructurado)? <https://www.ibm.com/es-es/topics/sql>

JavaScript.com. (s.f.). Learn JavaScript online – Courses for beginners. <https://www.javascript.com/>

Justinmind. (2025, 4 de febrero). Search UI: diseño de cajas de búsqueda, filtros y páginas de resultados. <https://www.justinmind.com/es/ui-diseno/pagina-de-resultados-de-filtros-de-busqueda>

Lenguajes de Programación. (2025). Lenguaje SQL: características actualizadas en 2025. <https://lenguajesdeprogramacion.net/sql/>

Lucidchart. (s.f.). Qué es un diagrama entidad-relación. <https://www.lucidchart.com/pages/es/que-es-un-diagrama-entidad-relacion>

MDN Web Docs. (s.f.). JavaScript. <https://developer.mozilla.org/es/docs/Web/JavaScript>

Microsoft. (s.f.-a). Microsoft Excel. <https://www.microsoft.com/es-es/microsoft-365/excel>

Microsoft. (s.f.-b). Microsoft Word. <https://www.microsoft.com/es-es/microsoft-365/word>

Microsoft. (s.f.-c). ¿Qué es SQL Server? Microsoft Learn. <https://learn.microsoft.com/es-es/sql/sql-server/what-is-sql-server>

Microsoft. (s.f.-d). Visual Studio Code. <https://code.visualstudio.com/>

Nasir, S. Z. (2021, 13 de noviembre). Databases and CRUD operations in C#. The Engineering Projects. <https://www.theengineeringprojects.com/2021/11/databases-and-crud-operations-in-c.html>

Ofner, W. (2018, 9 de enero). Repository and Unit of Work Pattern. Programming With Wolfgang. <https://programmingwithwolfgang.com/repository-and-unit-of-work-pattern/>

Oracle. (s.f.). MySQL. <https://www.mysql.com/>

Pencil Project. (s.f.). Pencil. <https://pencil.evolus.vn/>

Pontificia Universidad Católica del Perú. (2022, mayo). Lineamientos de accesibilidad web. Dirección de Comunicación Institucional PUCP. https://assets-global.website-files.com/60c8d29050a5cb271fbb9ea2/651d9b6a1db13a734a0b5586_5.%20Lineamientos%20de%20accesibilidad%20web.docx.pdf

Puig, P. (2026, 12 de marzo). Mejores prácticas de UX para los formularios (e incrementar la conversión). Cyberclick. <https://www.cyberclick.es/numerical-blog/mejores-practicas-de-ux-para-los-formularios-e-incrementar-la-conversion>

Rascia, T., & Tagliaferri, L. (2021, 25 de agosto). Understanding events in JavaScript. DigitalOcean. <https://www.digitalocean.com/community/tutorials/understanding-events-in-javascript>

Red Ninja. (2024, 23 de mayo). Metodologías de gestión de proyectos: Agile, Scrum y Kanban. Blog de Red Ninja. <https://redninja.es/blog/gestion-proyectos/metodologias-agile-scrum-kanban/>

Refactoring Guru. (2024). Patrones de diseño de software explicados. <https://refactoring.guru/es>

Ruiz Loro, J. (2021). Diseño de un sistema web para la gestión de biblioteca en la I.E. Federico Villarreal – Piura. Universidad Católica Los Ángeles de Chimbote. <https://repositorio.uladech.edu.pe/handle/20.500.13032/22813>

Sandaruwan, D. (2025, 13 de diciembre). DTO vs Entity: The Most Common Mistake in Spring Boot APIs. Medium. <https://medium.com/@dulanjayasandaruwan1998/dto-vs-entity-the-most-common-mistake-in-spring-boot-apis-9c015c905a72>

A Software Architecture for Inventory Management System - Scientific Figure on ResearchGate. (2012). https://www.researchgate.net/figure/internal-Structure-of-MCV-Architecture_fig2_291696392

Spring. (s.f.). Spring Boot. <https://spring.io/projects/spring-boot>

Stepanets, A. (2024, 16 de octubre). Indicadores de gestión: qué son y ejemplos. GanttPRO. <https://blog.ganttpro.com/es/indicador-de-gestion/>

Thymeleaf. (s.f.). Thymeleaf. <https://www.thymeleaf.org/>

Tutoriales Ya. (s.f.). MySQL Ya. <https://www.tutorialesprogramacionya.com/mysqlya/>

Visual Paradigm. (s.f.). What is Class Diagram? <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-diagram/>

YouTube. (s.f.). Patrones de interfaz de usuario (UI/UX). <https://www.youtube.com/watch?v=7VNOcBGMKrw>